

AI 能找到开发者所需的知识吗？开发者生态中面向 AI 的知识可得性的定义与度量

ANONYMOUS AUTHOR(S)

开发者日益广泛地使用AI Agent开展软件开发，从检索技术资料、生成和修改代码，到调用工具执行任务。Agent协助开发者完成这些任务，需要能够取得可用的技术知识。本文将这些条件概念化为面向AI的知识可得性，提出任务级协议及十一项指标，涵盖知识来源、获取成本和回答属性。该方法的历史应用覆盖类面向AI加速计算的软件开发任务；本次新增GLM-5.3、DeepSeek V4.1 Flash与Kimi-K3-2在三个任务、两个生态上的次探索性试跑，用于检查修订规则的执行与证据链。以下保留的历史分析区分了版本信息不清、核心操作正文未取得、正文可读但指导不足，以及主流程有据可依但部分参考资料不可得等情形。这些发现将开发者评估技术建议所需的信息与知识缺口联系起来，为技术知识供给及Agent对知识的获取与使用等方面的改进提供方向。本文贡献一个概念框架、一套可检查的协议及其案例应用，用于诊断Agent协助开发者完成任务时所依赖的知识条件。

CCS Concepts: • **Human-centered computing** → HCI theory, concepts and models; • **Software and its engineering** → Documentation.

Additional Key Words and Phrases: 面向 AI 的知识可得性；开发者生态；技术文档；信息寻径；AI Agent；测量方法

修订状态（-09-21）：评分规则、指标图与方法文献已更新。当前实验材料为三个任务的次试跑；全量实验部分待作者通知完成后更新，结果与结论届时讨论。

1 引言

在传统的软件开发方式下，开发者常常需要自行查阅文档、代码示例、搜索结果和社区解释，以决定下一步行动。关于机会式编程和开发者信息需求的研究表明，寻找与解释信息本身就是编程活动的一部分 [7, 15, 33]。有用的信息必须契合当前任务：命令需要正确的选项，API 示例需要对应的版本，错误解释需要提供足够的上下文来指导排查。因此，文档的存在只是其有用性的一个条件。

具备检索与工具调用能力的开发 Agent 为这一知识环境增加了一条协作路径。近期对GitHub项目的研究发现，编程Agent的采用已涉及不同成熟度和技术类型的项目 [29]。开发者既可以围绕报错或局部代码提出问题，也可以描述待完成的变更或结果目标，将资料查找、文档阅读、实现步骤组织以及部分代码修改委派给 Agent。为推进任务，Agent 可能自行在官网、文档、仓库与社区之间搜索、读取和整合材料，再起草回答、实现方案或代码改动。关于代码生成工具、对话式编程辅助和 Agent 支持的研究既描述了这种支持带来的机会，也揭示了理解和核验生成建议的困难 [5, 31, 36]。委托检索改变了支撑材料到达开发者的方式，但开发者仍需判断建议是否有可辨认的依据、是否符合本地环境，以及是否需要更多信息。Agent能否取得技术材料，是这些依据能否被呈现和检查的条件之一。测量这一获取过程，因而可以识别经由AI形成的指导所依赖的知识缺口。

Agent通常能够对开发问题给出回答，但回答是否准确、能否用于当前任务，受到其所取得知识的影响。例如，实现自定义算子时，Agent可能找到官方指南，却未能取得操作正文；排查错误时，也可能读到完整页面，但其中只有笼统的查看日志建议。在这些情况下，Agent仍可能依靠已有知识或推断继续作答，却缺少支撑具体步骤的外部依据。即使取得了详细说明，资料中的工具包与框架版本不兼容，也可能导致建议不适用。因此，评价不仅需要检查Agent给出了怎样的回答，还需要识别其知识依据缺在哪里，从而为资料完善和Agent反馈设计提供具体的改进方向。

开发者生态的建设者不仅需要知道Agent能否给出可用的技术指导，还需要判断哪些知识条件限制了这种指导，以及应当在哪里改进。相似的回答不足可能来自不同原因：官方资料未被发现、操作正文未能取得、已取得

内容缺少任务细节，或版本选择依据不明确；可用的回答也可能依赖替代来源，从而掩盖官方资料的缺口。仅观察最终回答或任务结果，难以区分这些情况。本文以开发者任务的知识需求为参照，比较不同生态对各类开发活动提供的知识支撑，并结合来源、获取事件和实际内容，识别资料供给、信息组织及来源互补方面的问题，为生态建设与Agent交互的改进提供依据。

研究缺口与本文的增量。现有研究分别讨论开发者如何寻找信息、文档如何支持API学习、检索增强系统是否取得相关上下文，以及Agent是否完成任务；这些工作提供了重要的概念和结果指标，但通常没有把“发现了什么、实际取得了什么、哪些任务需求得到支持、版本关系是否足以定版，以及最终回答还需怎样修正”放在同一条可追溯的证据链中。尤其是，检索结果、页面返回形态、来源归属和回答属性往往分别报告，因而难以判断问题究竟来自入口、正文交付、内容缺口、来源冲突还是回答组织。本文的额外贡献不是再提出一个答案正确率，而是提出面向AI的知识可得性构念，把上述条件按任务连接起来，并用冻结问句、实际工具返回和最终答案保存可复核证据。随后以CANN/CUDA案例检验这套诊断框架能否定位文档和Agent需要改进的环节；生态间差异是案例中的诊断输出，不是本文的唯一目标或总体排名。

我们将面向AI的知识可得性定义为：在明确的检索条件下，一个指定的Agent及工具配置能够发现、取得任务相关技术知识，并评估其适用性的程度。这是一个关系性构念：它涉及特定时点上的任务、知识环境，以及访问该环境的方式。我们通过证据记录协议和十一项指标将其操作化。这些指标区分来源条件、获取成本、回答检查，以及由M1-M8计算的综合置信度。

该方法的预期使用者包括产品经理、设计师、技术写作者和开发工程师等，他们参与开发者平台、技术知识资源或AI Agent的建设与维护。该方法旨在帮助他们定位技术资料在发现、获取和任务支撑方面的缺口，并明确知识供给与Agent反馈需要改进的环节。案例应用展示了可服务于这些用途的诊断区别；其在团队工作中的实际价值仍需评价。

本文提出两个研究问题。RQ1：如何定义并操作化面向AI的知识可得性，使任务特定的访问条件与证据缺口能够被系统记录和复核？RQ2：该方法在不同开发任务中揭示了哪些知识可得性模式，这些模式如何为文档与Agent设计提供依据？

我们通过一个方法框架及其在面向AI加速计算的软件开发任务中的应用来回答这些问题。历史案例覆盖类任务、条任务侧记录；当前修订另报告三个模型在A、I、E三个任务、两个生态中的次试跑。两批材料及量规分开记录，不把试跑写成全任务重跑。其中，类任务比较CANN与CUDA开发情境；一类迁移任务以ROCm/HIP为对照目的地，单独处理。案例应用将任务级测量连接到跨任务模式与设计启示。具体贡献包括：（）一个将知识条件与检索成功、答案正确性区分开的构念；（）一套任务级协议、指标定义和可移植的分析材料；（）具体案例与跨任务综合分析，区分不同断点，并将其连接到文档与Agent设计。

2 相关研究与构念边界

2.1 开发者信息寻找与文档

信息寻径理论将信息寻找行为与可用信息的结构和预期价值联系起来[27]。编程研究展示了开发者如何交织进行搜索、学习和实现，以及他们的问题如何依赖于正在开展的工作[7, 15, 33]。API学习中的困难也促使研究者关注技术接口周围的解释性资源[30]。Uddin与Robillard进一步指出，歧义、内容不完整与错误是API文档中的严重问题[35]；Aghajani等通过分析邮件列表、Stack Overflow讨论、问题单和拉取请求，形成了文档问题的分类[1]。Treude与Robillard则展示了从Stack Overflow提取API文档未包含的有用信息，以补充正式文档的可能性[34]。Maalej与Robillard进一步对API参考文档中的知识类型进行了分类[21]。这些研究表明，开发者需要什么信息，与正在完成的任务密切相关。因此，本文以具体开发任务为单位，考察Agent能否找到并取得所需资料，以及资料是否提供了完成任务所需的解释、参数说明和操作步骤。

本文方法不从机器访问情况推断人类可用性。对视障开发者的研究也表明，AI编程辅助的收益可能伴随建议过多和上下文切换等无障碍问题，用户体验仍需在具体使用情境中考察[9]。浏览器能够读取的页面可能对某个特定抽取工具不可访问；反过来，Agent容易抽取的文本也可能让人难以导航。我们考察的是受委托的知识获取路径所面对的条件。关于开发者时间、信任、满意度或任务完成情况的主张，需要另外的证据。

2.2 面向编程的 AI 支持

AI编程辅助研究区分了获得建议与理解、调整 and 检查建议。Vaithilingam等描述了理解、修改和调试生成代码时的困难[36]；Barke等区分了开发者使用代码生成模型时的加速与探索模式[5]；Ross等考察了围绕代码上下文开展的对话式辅助[31]。Liang等对名开发者的调查发现，输出难以满足功能或非功能需求、难以控制生成结果，是重要的使用障碍[18]。Mozannar等通过开发者活动分类分析与代码建议交互的行为和时间成本，说明评价也需要关注建议生成之后的使用过程[24]。在安全相关编程任务中，Perry等发现，使用AI辅助的参与者编写了更不安全的代码，却更可能认为自己的代码是安全的，提示主观信心与实际质量需要分别检验[26]。这些发现促使我们关注开发者考虑一项建议时能够获得哪些支撑信息。本文协议检查相关来源段落、版本条件和操作细节能否被取得并与建议关联，为研究开发者如何使用AI辅助提供知识条件层面的描述。

Agent能够交织推理与工具使用，而检索增强生成提供了将外部信息纳入生成过程的方式[17, 39]。WebArena、SWE-bench和SWE-agent在贴近真实的网页与软件工程情境中评估或开发Agent[12, 38, 40]。其任务结果是系统表现的重要证据。本文框架则对这些系统运行时面对的知识条件提供互补描述。失败可能涉及证据不可得、对可得证据的解释不充分，或执行不成功；本文方法直接处理其中第一类问题，并记录有助于区分这些情形的信息。

2.3 检索、归因与可得性构念

现有研究从多个层面评价AI系统。ALCE关注生成答案的流畅性、正确性和引用质量；RAGAs与ARES进一步评价检索上下文的相关性，以及答案对上下文的忠实程度[8, 10, 32]。AgentBoard通过细粒度任务进展率等指标分析Agent的行动过程，说明最终成功率之外的过程信息也具有评价价值[20]。在编程情境中，RepoBench分别评价跨文件代码检索、代码补全及二者组合的流程，明确区分了上下文获取与生成表现[19]。其对象是代码仓库中的补全任务；本文则检查开发任务所需的官方与第三方技术资料能否被发现、取得并提供充分支撑。这些方法为理解系统表现提供了基础。对于开发者生态的建设与维护，还需要将表现与具体知识条件联系起来：相关技术资料是否容易发现，所需正文是否实际取得，内容是否提供操作所需的细节，以及版本之间是否存在明确的选择与配套依据。本文围绕这些关系组织任务级测量，以区分需要改善获取路径、完善资料内容或明确版本关系的情形。

表1中的区分界定了这一构念的位置。知识可得性可以是基于证据作答的必要条件，却不足以保证正确性。Agent可能误解一个可得来源，也可能在未取得任何外部证据的情况下，凭先验知识给出正确答案。这些结果应保持可区分。

Table 1. 表 构念边界与相邻的评估对象。

评估对象	核心问题	与本文方法的关系
检索成功	是否返回了相关结果？	获取过程的一个条件；不能证明所需内容已被取得。
文档可用性	目标读者能否导航和理解这些材料？	相关的面向人类的属性，需要单独评估。

评估对象	核心问题	与本文方法的关系
面向 AI 的知识可得性	在所记录条件下，哪些任务相关知识能够被取得？	本文所操作化的构念。
答案正确性与归因	回答是否正确，其来源是否支持其中的主张？	可以使用已取得证据进行的下游评估。
执行成功	所提出的行动能否在目标环境中奏效？	需要执行或其他独立的结果证据。

3 方法框架

3.1 分析单位与任务需求

分析单位是一次任务侧知识获取过程：使用明确的 Agent 和检索配置，在特定技术生态中围绕一个开发目标开展知识获取。任务说明记录期望结果、初始材料、约束、版本依赖，以及支撑下一步行动所需的证据。例如，模型转换可能需要转换命令、输入形状说明、目标设备设置，以及检查生成产物的方法。这些需求指导检查；页面长并不自动意味着内容充分。

任务选择与抽样应得到明确说明，并服务于评价目的 [14]。生态评估可能寻求覆盖不同工作流；文档团队的审计可能聚焦反复出现的支持问题。若无进一步的抽样证据，两者都不能说明这些任务在总体中的发生频率。跨生态应用时，分析者应记录初始条件与任务范围的差异，而不是假设名称相近就意味着任务等价。该框架也可用于同一生态内部、不同版本之间或不同检索配置之间，只要比较条件得到明确说明。

图 1 将各项测量放在一条可观察的工具调用循环中。它是一个分析模型，而非对模型隐含推理过程的重建：Agent 先搜索候选来源、选择 URL、读取正文或记录失败，再汇总证据并检查任务充分性与版本适用性。证据不足而仍有可检索空间时，循环回到查询精化；模型先验作为独立分支登记；当前协议还在检索前冻结无工具答案以供 M7 核查。因此，先验知识不自动视为可追溯证据。

3.2 知识来源与任务支撑

该框架考察三类知识来源：官方资料、第三方资料和模型自带知识。官方资料与第三方资料的区分依据发布者身份确定。例如，厂商发布的文档、代码仓库和博客属于官方资料，独立作者提供的教程与实践经验属于第三方资料；托管平台本身不决定来源归属。

对官方和第三方资料，框架考察 Agent 能否找到相关来源、能否实际取得内容，以及取得的内容能否支撑当前任务。具体检查包括：是否包含所需的操作步骤、参数解释或诊断信息，是否明确适用版本及组件配套关系。对于模型自带知识，当前协议在检索前冻结无工具回答，再检查其正确性与任务覆盖，并与实际取得的外部资料分别呈现。

将三类来源结合起来，可以分析任务所需知识由哪些来源提供、哪些需求仍缺少支撑，以及不同来源能否相互补充。例如，官方正文未能取得时，第三方资料是否提供了可用的操作说明；多个来源都有相关内容时，其版本与配套信息是否一致。获取次数与失败尝试则用于描述取得这些知识所需的成本。

3.3 证据记录

证据记录将每项判断与其依据联系起来。最低限度的字段包括任务问句与初始条件、已知的 Agent 与工具配置、采集时间、查询和选定 URL、发布者与来源角色、返回内容或相关片段、访问结果、适用版本、内容所支持

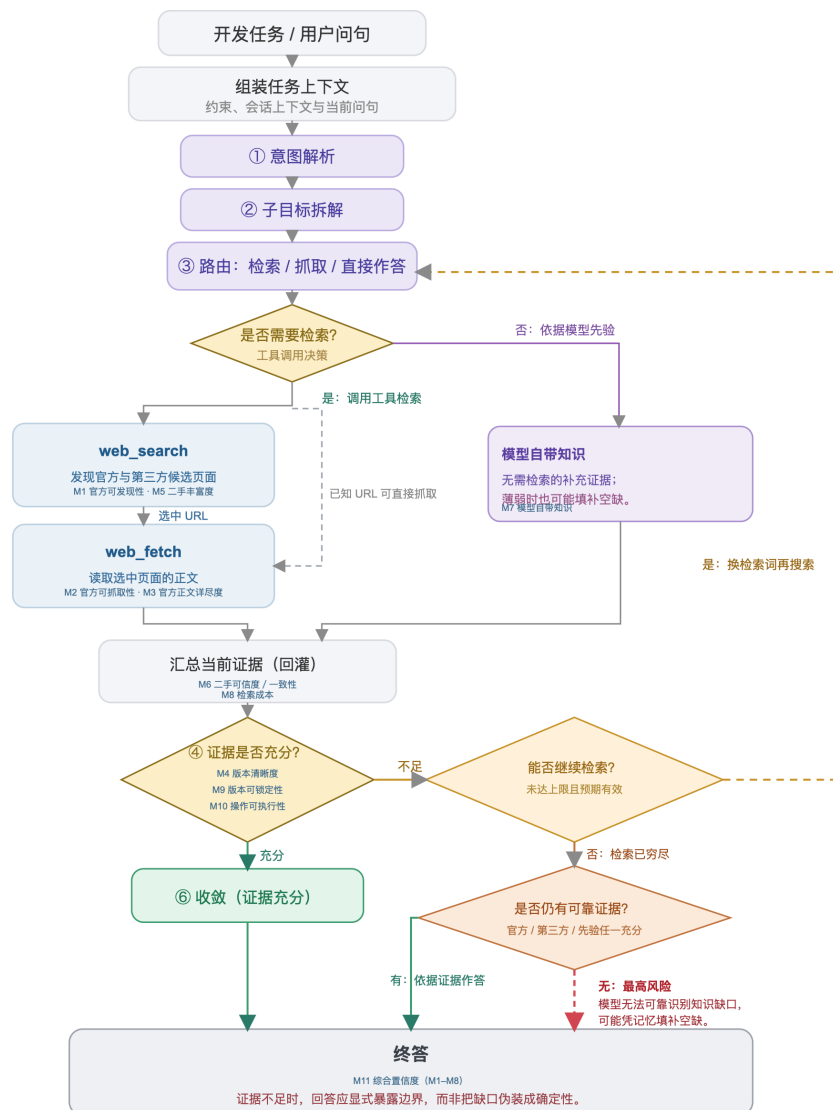


Fig. 1. Agent 以任务与上下文为起点，经由意图解析、子目标拆解和路由判断，选择检索或模型先验；检索分支依次搜索、选取 URL、抓取正文或直接抓取已知 URL，再汇总证据并判断充分性。证据不足时可重试；检索穷尽时会区分是否仍有可靠证据。

的任务需求、获取次数，以及编码决定及其理由。另设字段标明条目属于直接记录、后续编码、估计，还是不可获得。

这一区分可以避免一种常见的报告错误：仅仅因为脚本为某个解释赋予了数字，就把解释转化为观测。抽取失败反映工具与来源交互的结果；将某段解释判为充分，是对照任务需求作出的判断；估计模型对某个工具包的熟悉程度，则属于推断，除非有单独测试为其提供支持。数值编码不会消除这些差别。

4 操作化与测量协议

4.1 十一项指标及其证据角色

本方法保留官方资料、第三方资料和模型自带知识三类来源，以及发现、获取、内容支撑、版本条件、成本与回答属性的评价目的。当前操作化采用本次试跑预先冻结的规则，不沿用已安装旧版Skill的分档。表区分每项指标的评价对象。

4.1.1 指标选择逻辑与相互关系. 11项指标不是互相独立的“分数清单”，而是从任务知识链条逐层拆出：官方渠道由M1–M4覆盖发现、实际取得、内容详尽度和版本适用关系；第三方渠道由M5–M6覆盖来源数量与关键说法的证据质量；模型先验由M7单独记录；M8记录取得这些材料的检索与获取成本；M9–M10检查外部知识进入最终回答后的版本定版与操作补改；M11只把M1–M8透明地汇总为一个综合指数。这样安排是为了让每个分数对应一种可修复的缺口，而不是重复测量同一件事。

指标之间的边界也预先固定。M1问“能否找到官方来源”，M2问“实际取得的官方正文有多少”；M3问正文是否覆盖任务内容，M4问资料是否给出必要的版本—组件关系；M5是相关独立第三方来源的数量，M6是关键说法能否被支持或反驳；M7不使用本轮取得的资料，M8只记录实际发生的检索轮次、获取调用与失败次数；M9和M10只看最终答案，分别对应定版和操作补改。因此，正文很长不自动提高M3，来源很多不自动提高M6，官方资料清楚也不自动提高M9。本文没有把人类导航易用性、开发者满意度/信任、端到端硬件执行成功率、延迟或token成本并入这项，因为这些问题需要不同的观察单位和独立证据；遗漏它们是构念边界，而不是暗示它们不重要。

Table 2. 表 指标的评价对象与证据边界。M1–M10采用有序五档，M2另保留逐文档等权均值；未知、不适用与受阻不是低分。

编号	指标	证据与解释
M1	官方可发现性	统一搜索预算内，首次发现相关官方来源的查询次序与列表位置；改写查询本身不另扣分。
M2	官方可抓取性	每个独立官方文档最终实际取得的表示及正文完整性；先区分错误、框架、摘要和直接正文，再核对正文边界。
M3	官方正文详尽度	已取得官方正文对原题所需解释、步骤、参数与约束的说明程度；无正文可评时记受阻。
M4	版本清晰度	已取得官方资料能否提供完成版本选择所需的适用关系；不按版本或页面数量扣分。
M5	二手丰富度	预算内发现的相关独立第三方来源数；另记首次发现位置与查询数，不以取得正文为计数前提。
M6	二手可信度 / 一致性	对会改变操作、解释或版本判断的关键说法逐条核对支持、反驳与未知，并检查独立来源互证。
M7	模型自带知识	冻结问题检索前无工具回答，再核查正确性与任务覆盖；不以自报信心或推测训练数据密度计分。

编号	指标	证据与解释
M8	检索成本	实际搜索查询数S与获取调用数F之和 $C=S+F$ ；失败调用计一次，不再额外惩罚，低成本得高分。
M9	版本可锁定性	最终答案对原题所需组件给出的有据版本范围或具体版本；与官方资料的版本清晰度分别评价。
M10	操作可执行性	最终答案中的命令、代码、参数与前提还需多少补充或修正；实际运行状态另列。
M11	综合置信度	由M1–M8按固定公式构成的连续–100综合指数；M9/M10独立报告，不解释为答对概率。

4.1.2 指标的文献依据与本文操作化.

指标	文献提供的概念或方法参照	本文的测量定义、证据规则与判分约定
M1 官方可发现性	排名检索与交互式信息检索评价 [14, 22]	以首次相关官方命中为对象；按首次命中的查询轮次和前五位位置映射–5分，查询上限作为试跑条件单独记录。
M2 官方可抓取性	数据质量维度与正文 / 模板区分 [16, 37]	以独立官方文档的最终返回表示为单位；按无目标内容、框架、摘要 / 摘录、带缺损正文、未观察到缺损的正文判定–5分；完整性另行核验，重试合并后独立文档等权平均。
M3 官方正文详尽度	情境化信息质量与API文档知识类型 [21, 35, 37]	以冻结任务需求为覆盖清单；从概述、零散片段、完整段落、主流程到全部所需解释 / 步骤 / 参数 / 约束判定–5分；没有正文记 blocked。
M4 版本清晰度	API文档中的适用关系、上下文和约束 [21, 35]	评价版本—组件关系是否可用于选择版本；从无版本信息、无选择依据或冲突，到部分关系、合并约束和官方直接规则依次判定–5分；不按版本数量扣分。
M5 二手丰富度	正式文档与第三方补充来源的互补性、交互式检索来源计数 [14, 34]	统计预算内相关且独立的第三方来源；合并转载和镜像，按、–2、–4、至少条映射–5分；不推断生态总来源量。
M6 二手可信度 / 一致性	逐条事实核验、检索上下文与回答评价的区分 [8, 23]	以会改变操作、解释或版本判断的关键说法为单位；按未解决矛盾、无可追溯支持、部分支持、全部支持无独立互证、全部支持且有独立互证判定–5分；没有第三方材料记N/A。

指标	文献提供的概念或方法参照	本文的测量定义、证据规则与判分约定
M7 模型自带知识	无检索知识表现与事实核验 [13, 23]	只评价首次工具调用前的无工具回答；按正确知识与原题流程 / 参数 / 约束覆盖程度判定-5分，自评信心不计分，不能由检索后内容补高。
M8 检索成本	交互式检索过程与成本记录、聚合指标披露 [14, 25]	令 $C = S + F$ ，只计实际派发的搜索和获取调用，失败调用计一次、拦截请求不计；按-2、-4、-6、-8、至少次反向映射-1分。
M9 版本可锁定性	信息质量、API文档适用关系和选择条件 [21, 35, 37]	只评价最终答案对原题所需组件的版本及证据支持；按版本缺失、无适用依据、部分组件、全组件范围、全组件具体版本判定-5分；无版本要求记N/A。
M10 操作可执行性	量规判定与API文档中的操作细节 [21, 28, 35]	只评价最终答案还需多少补写或内容修正；按目标描述、操作骨架、需补改方案、仅替换已说明环境值、无需补改判定-5分；实际运行状态另列。
M11 综合置信度	综合指标的聚合、权重披露与效应度解释 [2, 6, 25]	用预先固定的M1-M8公式形成连续-100指数：渠道内乘法、跨渠道noisy-OR及.30 / .10修正；M9/M10独立报告，指数不解释为答对概率。

文献支持测量对象、证据边界、事实核验和聚合假设，但没有直接验证本研究的阈值、权重或概率解释。

M2回答“实际取得什么”，M3回答“取得的正文说明是否详尽”，M4回答“官方适用关系是否足以选择版本”，M9回答“最终答案是否有据定版”，M10回答“操作指导还需怎样补改”。这些对象不能互相替代。正文包含代码不证明全文完整；版本范围有据不等于答案已锁定具体版本；内容核查也不等于实际执行成功。

4.2 协议的应用

第一，固定任务和条件。每个分析单元由任务、生态、实际请求模型、轮次及run_id标识。预先保存两侧问句、初始条件、需求清单、版本选择范围与需定版的组件，记录模型及工具身份、查询深度、搜索 / 获取预算、时间与停止条件。任务配对只能说明共同开发意图，不能保证难度严格相同。工具未暴露的参数标记未知。

第二，冻结检索前回答并记录获取。模型在接触本轮搜索结果前完成问题无工具回答，供M7后评。随后由客户端保存真实工具请求与返回，包含每轮查询及全部有序结果、未打开或重复条目、URL、标题、摘要、位置与可得日期。获取前明确目标为完整文档或具体章节，不得事后缩小成恰好返回的片段。保留原始资源格式、工具返回表示、正文或错误信息；没有正文不能单独证明SPA、robots或动态渲染原因。

第三，关联需求、来源与最终回答。记录原始障碍、后续替代路径、恢复状态与最终仍未满足的需求。取得其他正文不能删除早期失败，也不能自动证明任务已解决。M3/M4依据实际取得的官方资料；M9/M10依据实际最终答案，引用来源段落不能代替答案原文。

第四，分层后评并保留修订。每次运行保存原始过程记录和评价记录；自动后评原件、复核后的派生评价和事后参照另存并关联，不互相覆盖。分数须附事件编号、可定位引文和判断理由；引文匹配仅证明文字位置，

不证明技术正确性。事后取得的同URL参照可用于审计运行时返回的完整性，但必须标记时间和用途，不能加入模型当时的知识上下文。

第五，报告分数及不可评分状态。能判定档位时给点值，存在明确上下界但不能定档时保留区间及原因；未评价、证据不足、缺测、不适用和受阻分别保留。M2不能删掉未知文档后取平均。M11的当前计算器要求M1-M8全部为可用点值；其余记录保留待定，不补零，不据少数可算记录制作总体排名。记录实际运行通过、失败、仅内容核查或未验证，避免把M10内容评分写成执行成功。

4.3 评分规则与综合置信度

以下图示概括冻结规则；附录A列出完整五档和判定边界。这些有序档位是研究操作化约定，并不表示相邻等级具有已验证的等距含义。

量规与权重如何形成。量规先从构念边界和冻结任务需求列出每项需要观察的对象，再为每项规定证据边界、缺失状态和可修复的判定条件；随后用边界案例检查容易混淆的分支，冻结规则，最后用结构、引文定位和算术检查验证执行一致性。文献为“应测量什么、如何区分证据与推断、如何披露聚合假设”提供参照，但没有一篇文献直接给出本研究五档阈值。M11在渠道内采用乘法，是为了保留发现、取得或支撑任一环节不足的限制；渠道之间采用噪声OR，是为了表达官方、第三方和模型先验可能互相补充。版本系数.30和成本系数.10是预先指定的工作基线，作为对主要支撑渠道的有限修正，不是从本批结果拟合出来的最优权重；敏感性分析和未校准限制因此单独报告。这样可说明分数是如何形成的，也避免把精细的数字写成已经验证的客观量表。

官方资料 (M1-M4)。M1按首次相关官方命中的查询轮次和前五位位置判定：为用完规定预算仍未命中，为第次或之后才首次命中，为第次首次命中，为第次的第-5位命中，为第次的第位命中。本次最多四次查询是试跑条件，不是指标的通用定义；先核实官方身份和任务相关性，再使用保存的有序排名，不能用最终答案中的引用顺序替代搜索排名 [14, 22]。M2只评价每个独立官方文档的实际返回表示及可观察缺损：为失败、拒绝、错误页或没有可识别目标内容，为只有标题 / 导航 / 元数据，为明确摘要或摘录，为直接正文但有明确截断、缺页或目标边界缺失证据，为直接正文且未观察到这些缺损。摘要内含代码不升级，静态页与服务端渲染页按同一内容标准判断。分不再声称已经证明全文完整；完整性另存为独立核验状态，未知本身不把分数改成-5。重试按同一URL合并，不重复计数；不同独立文档等权平均，保留原始精度，不能删除未知文档后求均值 [16, 37]。

M3只按原题列出的需求核查已取得正文的详尽度：为只有概述，为零散参数或片段，为有完整段落但关键步骤、参数或约束仍缺，为主流程、参数和前提齐全但题目涉及的分支或约束仍缺，为原题所需解释、步骤、参数和约束均已说明。它不增加题外的穷尽要求；没有取得正文记blocked，已取得但尚未完成核查记待复核。M4依次区分：无版本信息；有版本但无选择依据或存在未消解冲突；部分适用关系明确但缺必要对应；需要合并有依据的官方约束才能确定；官方已用表格或规则直接明确适用关系、可以直接查表或套用。可选结果是单个版本或有依据的范围；检索成本归M8，最终答案的定版粒度归M9 [21, 35, 37]。

指标 / 得分	1	2	3	4
M1 官方可发现性	预算内未发现	第3次及之后 查询首次发现	第2次查询 首次发现	首查第2-5条
M2 官方可抓取性	未取得可识别 目标内容	仅标题、导航 或元数据	明确摘要 / 摘录 未取得直接正文	直接正文 有截断 / 缺失证据
M3 官方正文详尽度	只有概述 无相关细节	零散相关细节 未成完整段落	已有完整段落 关键内容仍缺	主流程与前提齐 所需分支仍缺
M4 版本清晰度	无版本信息	有版本，但无选择依据， 或官方说明冲突且 无法消解	已明确部分适用关系， 但仍缺少完成版本选择 所需的对应关系	需自行合并官方约束， 才能确定适用版本

M2：按独立文档最终状态等权平均；重试不重复计数；完整性未知保留4-5，不删除未知项。
M3/M4：只看原题与实际取得的官方内容；无正文记受阻；版本多不扣分。

冻结规则：2026-09-21 · 完整判定见附录A · 五档为研究约定

Fig. 2. 官方资料评分 (M1-M4)：按发现位置、取得表示与可观察缺损、内容详尽度及版本适用关系分别判定。

第三方资料 (M5-M6)。M5对预算内发现、去重且相关的独立第三方来源计数：、-2、-4、、至少条对应-5分。官方仓库和官方论坛按冻结渠道分类排除；转载和镜像不算独立来源，相关性未定时保留计数争议。它测量的是补充来源的数量，不是网页数量或生态总量。M6按关键说法的支持、矛盾和独立互证判定：为存在未解决的关键矛盾，为有可识别说法但没有可追溯支持，为部分关键说法有支持，为全部关键说法有支持但没有独立互证，为全部关键说法有支持且有独立互证。作者身份、第一手材料和可复核实验证据是核验依据；平台名、文章新旧或转载数量不能直接换算分数。没有第三方材料时M6记不适用；“未核验”不能直接当成“错误” [8, 14, 23, 34]。

模型自带知识 (M7)。M7只评价首次工具调用前保存的无工具回答及其对原题的覆盖：为没有可核验的正确任务知识或关键主张被可靠证据否定，为只有正确概念而没有具体方法或解释，为有正确方法 / 解释片段但关键流程或解释环节仍缺，为正确的主流程 / 解释已给出但题目涉及的参数或约束仍需补充，为原题所需流程、解释、参数与约束均已准确覆盖。自评信心只作记录，不计分；不能从无检索表现倒推训练数据密度，也不能用检索后的内容补高先验分。旧记录没有检索前回答时记缺测，正确性未核清时记待定 [13, 23]。

检索成本 (M8)。令 $C = S + F$ ，其中 S 为实际派发的搜索查询数， F 为实际派发的获取调用数，包括失败调用； $C = 1-2$ 、 $3-4$ 、 $5-6$ 、 $7-8$ 与 $C \geq 9$ 分别得、、、分。预算拦截后未派发的请求不计，失败调用只计一次且不额外扣分； $C = 0$ 或异常中断不自动获得高分。耗时和可得token用量另存，因为调用数不是完整计算成本，也不假定搜索与获取等时 [14]。

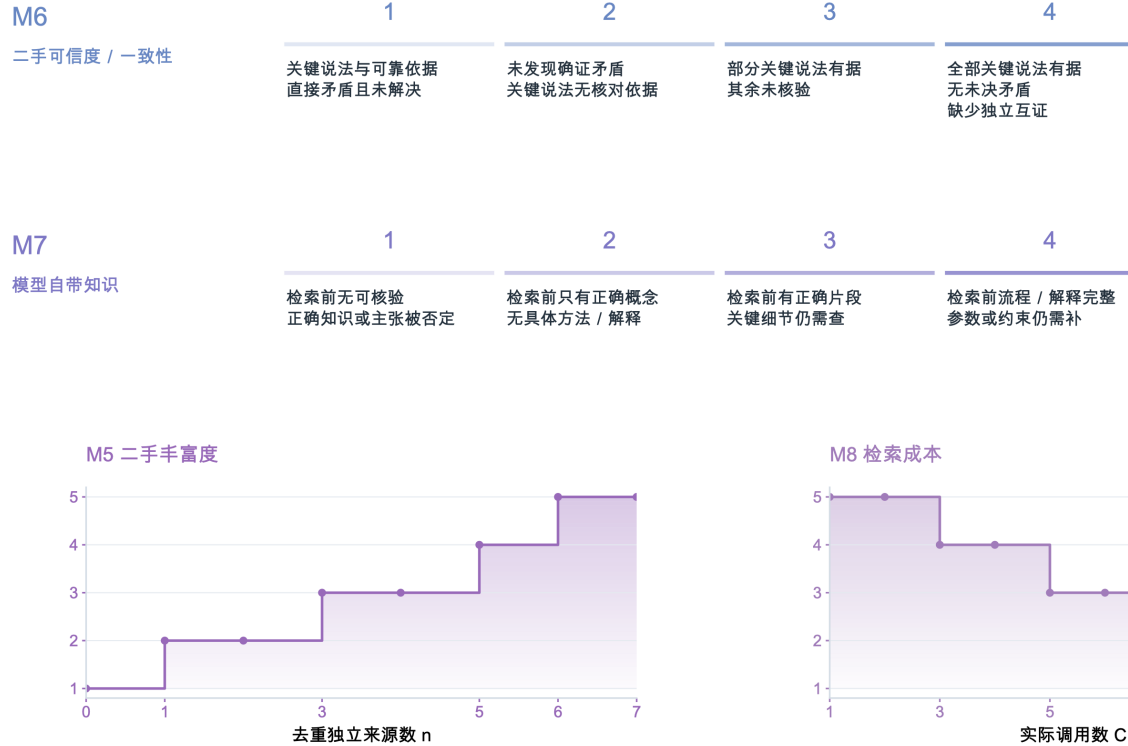


Fig. 3. M5–M8评分：独立来源计数、关键说法可信度、检索前知识表现与实际查询 / 获取成本。

回答产出 (M9–M10)。M9依次区分：缺少原题所需版本，有版本但适用性无据或关键定版与证据矛盾，只有部分所需组件有版本依据，所有组件都有据的适用范围但至少一项未锁定具体版本，所有组件都有据的具体版本且无未消解配套冲突。范围答案可以正确，但锁定粒度低于具体版本；不为提高得分臆定版本。M10依次区分：只有目标描述或示意，只有操作骨架且关键操作无法确定，有具体方案但需要补写或修正，指导完整、只需替换已明确标识并解释含义的用户环境值，指导和适用前提明确、无需补写或修正即可执行。分不能包含内容错误修正，未上硬件运行不是 / 分的分界；两项都只依据最终答案及其引文，纯概念任务的M10或没有版本要求的M9记不适用 [21, 28, 35, 37]。

指标 / 得分

1

2

3

4

M9

版本可锁定性

无原题所需版本

有版本但适用性无据
或关键定版矛盾部分所需组件有据
其余未确定所有所需组件
均有据范围
至少一项未定具体版

M10

操作可执行性

只有目标 / 示意
无可执行方案仅操作骨架
关键操作无法确定具体方案需补充
或修正才能执行完整指导，仅替换
已标明且有说明的
用户环境参数

M9：先固定原题需要定版的组件；无版本要求记N/A。

M10：4分不能包含内容纠错；运行状态另记，未实际运行不是4/5分界。

冻结规则：2026-09-21 · 完整判定见附录A · 五档为研究约定

Fig. 4. 版本可锁定性 (M9) 与操作可执行性 (M10) 。

综合置信度 (M11) 。令 $x_i = M_i/5$ 。渠道内部采用乘法，以保留各环节对该渠道支撑的限制；渠道之间采用噪声OR形式，表达任一来源可能补足其他来源的作用：

$$OFF = x_1 x_2 x_3, \quad SEC = x_5 x_6, \quad OWN = x_7,$$

$$K = 1 - (1 - OFF)(1 - SEC)(1 - OWN),$$

$$M11 = 100K(0.7 + 0.3x_4)(0.9 + 0.1x_8).$$

固定版本系数.30、成本系数.10，保留原始计算精度、展示一位小数。M11为连续百分制，不划五档，不采用 $(M_i - 1)/4$ 归一化或三渠道均值。M9/M10独立报告。该指数未经概率校准，噪声OR形式不证明渠道独立，也不允许将其解释为答对概率；它是透明的聚合报告值，不是经过效度验证的单一量表分数 [2, 6, 25]。

未知、不适用或受阻不能自动补零；包括M3受阻时，也不能自动令OFF=0。冻结计算器尚未实现“已确认渠道不可用”的特殊编码，M1-M8任一输入不能确定为可用点值时，本轮M11保留待定。M2等分项的区间仍完整呈现，不据此冒充已计算的M11点值。

M11 综合置信度



Fig. 5. M11连续百分制色带：颜色由浅至深表示-100连续分数；计算公式见正文。

系数作为预先指定的工作基线保留，不宣称最优。另一个独立的系数拟合批次完成次运行，开发集纳入/32、留出集纳入/16；主分析选择原系数，事后答案事件编号修正的敏感性分析选择不同候选，未得到稳定优于基线的证据。因此不采用上一轮拟合值。该批次与本次三模型次试跑分别报告，不合并样本或把自比较结果当作效率证据。模型选择与留出评价应分开，综合指标的权重假设及敏感性应披露 [11, 25]。

排名检索评价支持固定检索深度、保留位置与相关性判断 [22]；正文抽取研究支持将正文与模板、导航分开 [16]；信息质量与API文档研究支持区分可访问性、情境相关的内容质量和任务所需细节 [21, 35, 37]。这些原则分别帮助界定M1、M2和M3/M4的对象，不提供本文的五档阈值。

第三方来源补充正式文档的研究支持检查来源互补 [34]。逐条事实核验为M6和M7的可核查陈述提供方法参照；检索上下文与最终回答分开评价则帮助保持M9/M10的边界 [8,23]。M7的五档无工具任务知识表现和M10的补改程度均是本研究操作化；事实支持不能证明任务完整性或代码运行成功。交互式检索评价支持记录过程与成本，但不规定S+F及其分档，也不证明两种调用等时 [14]。

综合指标手册支持披露聚合与权重假设及检查敏感性，不验证本研究噪声OR公式、.30/0.10系数或指数的概率含义 [25]。测量开发与效度文献支持明确评价领域、检查内容覆盖，并为特定解释和用途积累证据；本框架包含多个不同对象，不作为单一心理潜变量量表处理 [2,6]。当前文献核对范围按文献账本记录，包含题录、摘要、选定章节或相关全文段落，不声称全部阅读全文。文献支持原则、日志检查通过以及多模型观察相近，均不等于具体阈值、系数或整套量规已经有效。

5 案例应用：用 CANN 与 CUDA 检验知识诊断框架

本案例把 CANN 与 CUDA 作为压力测试场景：两个生态在文档、版本和工具链条件上不同。其目的不是给出脱离任务范围的生态总体排名，而是检验这些指标能否定位可修复的知识缺口，并把缺口连接到文档和Agent的改进动作。

5.1 任务覆盖与比较范围

当前规则的执行材料来自年月日试跑：GLM-5.3、DeepSeek V4.1 Flash与Kimi-K3-2分别在A模型转换、I版本配套、E错误诊断三个任务上执行CANN/CUDA两侧问题，共模型×3任务×2生态=18次运行，每格一次。本次沿用历史问句并预先选择任务类型，用于检查规则执行和证据链；不是全部个历史任务的重跑，也不是独立留出验证或生态总体抽样。

Table 3. 表 本次试跑使用的三个任务问题对；每侧均有六项预先冻结的需求。相同开发意图不代表严格等难，A两侧的输入起点和转换范围仍不同。

任务与共同开发意图	CUDA侧问句	CANN侧问句
A：模型转换与部署	训练好的 PyTorch 模型怎么导出成 TensorRT engine 部署？给出 PyTorch→ONNX→TensorRT 完整步骤和命令。	ONNX 模型怎么在昇腾上转成 .om 离线模型部署？给出 ONNX→ATC→.om 完整步骤和命令。
I：版本配套	我的 PyTorch 跟 CUDA/cuDNN 版本对不上，怎么查 PyTorch、CUDA toolkit、cuDNN、驱动之间的版本配套关系？	我装了 torch_npu 报版本不匹配，怎么查 CANN、torch_npu、固件驱动、PyTorch 之间的版本配套关系？
E：错误诊断	我的 PyTorch/CUDA 程序报 RuntimeError: CUDA error: device-side assert triggered，怎么定位是哪一行、哪个 kernel 出的问题？	我在昇腾上跑训练/推理，报 EZ9999 之类的错误码，怎么定位根因、怎么排查？

历史审计于年月-11日开展，覆盖类任务、条任务侧记录；本文报告的模型为GLM-5.3、DeepSeek V4.1 Flash和Kimi-K3-2。其中对属于CANN/CUDA主比较；G的对照侧为ROCm/HIP，单列处理。这些历史记录保留原始身

份与数值，并与按新规则获得的三模型记录分开保存，不直接合并。下文既有历史结果和结论尚待结合新材料讨论。

5.2 任务实施与数据分析

本次通过WorkBuddy CLI请求glm-5.3、deepseek-v4.1-flash与kimi-k3-2，客户端返回与请求一致。显示名称统一写作GLM-5.3、DeepSeek V4.1 Flash和Kimi-K3-2；底层模型快照不保证不可变。三模型使用WebSearch和WebFetch以及相同冻结任务包、评分规则和预算。每次最多次搜索与次获取，客户端调用前拦截预算；最大轮、超时秒，生成effort设置为low。搜索提供方由客户端管理且未独立锁定，语言 / 区域、返回数量配置与temperature未暴露；实际有序返回全部留存。本次观察对象是模型与工具配置的组合，不能将所有差异归因于模型本身。

各运行使用新的临时工作目录和会话，并设置禁用记忆的环境标志；这些措施不是操作系统级隔离保证。执行顺序及三路并发由批次调度记录。模型先输出检索前答案，再调用工具，最后输出最终答案和停止原因。原始过程日志与每次评价分别保存；全批实际次搜索、次获取，留存条有序搜索结果。未发出的超预算请求与实际调用分开计数。次记录均通过已有日志对应、预算与输入哈希检查；这类检查仅支持记录完整性，不证明技术正确性。

所有运行均按统一后评流程处理，将来源 / 检索前答案与最终答案放在分开的评价上下文中。当前有效份后评，另保留次失败尝试；没有为获得偏好的分数重跑任务。一条检索前答案缺少结束标签，按首次工具调用前的真实事件恢复，只采用补评中的M7。原始自动后评、引文和规则复核后的派生评价以及评价修订记录分别保留；自动后评不等同于独立专家验证。

M2复核对已识别的官方获取目标逐文档登记，合并同URL重试后形成个“运行内独立文档”观测，其中包含跨运行重复访问。个观测的全文匹配依据来自运行后取得的同URL HTTP参照；其用途是审计原始返回的完整性，不是模型在运行中读到的新证据。其他观测按错误、页面框架、摘要或完整性待定分别保留。未知文档参与区间聚合，不删除后平均。

E任务的M4版本选择范围未在采集前固定，后评出现有分与不适用的分歧。因此本次对E的六条M4统一暂缓比较，保留原判断与原因，不强制改成同一分数。这是协议适用范围的遗漏，不能归因于模型能力。M9等仍有适用性或证据待核项，逐项显示，不以强行补分消除差异。

分析先按模型内问题两生态查看分项证据、区间和缺失原因，再检查跨模型是否遇到相同障碍类型及其恢复路径；未访问相同来源记录未观察，不能称没有该问题。三个任务和各模型单次运行不足以建立总体排名或测量效度。目前仅18份记录的M1-M8全部具备可用点值，故不制作三模型M11综合排名；M9/M10继续独立报告。没有硬件执行，M10仅涉及操作内容核查。

历史结果保留区（待讨论）：以下结果、图-15及相关讨论沿用旧审计材料和旧量规，数值未按-09-21规则重算，不代表本次次试跑。其解释与结论的取舍将在讨论新证据后修订；不能用新图-5反推旧矩阵分数。

5.3 任务级评价结果

主比较包含项CANN任务：项取得核心正文，项取得部分内容，项未取得核心正文。对应的项CUDA任务均取得核心正文。图-8呈现全部十一项指标评分，并按环境与安装、算子开发、训练、推理与部署、性能与优化、调试和迁移等工作流分组；G 作为单独标注的 CANN / ROCm-HIP 迁移类比呈现，不计入 CANN/CUDA 汇总。

图-8呈现M1-M10的有序评分，以及由M1-M8计算的综合置信度。图补充正文获取状态，使读者能够结合获取结果理解相应评分。

A. 官方来源条件

按 workflow 组织的指标评分。每项任务均列 CANN 与 CUDA；G 的第二列为 ROCm/HIP。

任务	M1 官方发现		M2 官方可获取		M3 官方详尽		C
	CANN	CUDA	CANN	CUDA	CANN	CUDA	
环境与安装							
I 版本配套	3	3	4	5	4	5	
J 安装与环境	5	4	4	5	5	5	
K 容器环境	5	5	5	5	5	4	
算子开发							
C 算子库选型	4	5	3	5	3	5	
D 自定义算子	2	5	2	5	—	5	
L 算子精度	2	2	4	5	5	5	
M 动态形状 / Tiling	2	5	4	5	4	5	
N 算子融合	5	2	4	5	5	4	
O 注册与框架集成	2	5	4	5	5	5	
训练							
F 分布式训练	4	5	4	5	4	5	
P 混合精度	3	5	4	5	4	5	
Q 显存 / OOM	5	5	4	5	5	5	
R 训练精度与收敛	3	3	4	5	5	4	
推理与部署							
A 模型转换 / 导出	4	5	4	5	4	5	
H 量化	3	5	4	5	3	5	
S 推理部署	5	5	4	5	4	5	
T 动态推理	3	3	4	5	5	5	
性能与优化							
B 性能定位	4	4	4	5	4	5	
U 访存与利用率	4	5	4	5	4	5	
V 计算与传输重叠	4	5	4	5	4	5	
W 多卡通信	2	5	4	5	3	4	
调试							
E 错误码排查	3	4	4	5	2	5	
X 内存越界	5	5	4	5	5	5	
迁移							
Y 跨芯片迁移	2	5	3	5	4	5	
Z 编程概念对照	5	5	4	5	5	4	
迁移类比 (不计入 CANN/CUDA 汇总)							
G 迁移类比	4	5	4	5	4	5	

M1-M10 为有序评分（1-5；高值更有利；M8 为逆向成本评分）。“—”表示官方正文详尽度未观测。
* M7 为模型自带知识估计。† M11 为由 M1-M8 汇总的综合置信度。G 的 CUDA 实为 ROCm/HIP，且不计入 CANN/CUDA 汇总。

Fig. 6. 完整指标矩阵 (/3)：按 workflow 分组呈现每项任务的 CANN 与 CUDA M1-M4 评分，即官方可发现性、官方正文可获取性、官方正文详尽度和资料版本清晰度。G 的第二列为 ROCm/HIP。

在对任务中，综合置信度均值为 CANN $\bar{x} = 0.731$ 、CUDA $\bar{x} = 0.922$ 。分项统计进一步显示，两侧在正文详尽度、版本清晰度、检索与获取成本，以及回答步骤可操作性方面呈现不同程度的得分差异。表汇总这四项指标的均值和标准差；下文结合评分分布与具体材料，解释这些差异所对应的知识条件。

B. 第三方、先验与获取条件

按 workflow 组织的指标评分。每项任务均列 CANN 与 CUDA；G 的第二列为 ROCm/HIP。

任务		M5 第三方数量		M6 第三方可信		M7 自带知识估计*		M8 第三方数量
		CANN	CUDA	CANN	CUDA	CANN	CUDA	
环境与安装								
I	版本配套	3	2	4	4	2	4	3
J	安装与环境	3	3	4	4	4	5	3
K	容器环境	2	3	2	4	3	5	3
算子开发								
C	算子库选型	3	5	4	4	3	5	3
D	自定义算子	3	4	2	4	2	5	3
L	算子精度	3	2	4	4	3	4	3
M	动态形状 / Tiling	2	2	4	4	2	5	3
N	算子融合	2	2	3	4	2	4	3
O	注册与框架集成	3	2	5	4	3	5	3
训练								
F	分布式训练	3	3	3	4	3	4	3
P	混合精度	3	3	4	4	4	5	3
Q	显存 / OOM	3	3	4	4	3	5	3
R	训练精度与收敛	3	3	4	3	3	5	3
推理与部署								
A	模型转换 / 导出	4	4	4	4	3	4	3
H	量化	4	3	3	4	3	4	3
S	推理部署	2	3	4	4	2	4	3
T	动态推理	3	2	5	4	3	5	3
性能与优化								
B	性能定位	3	4	3	4	3	4	3
U	访存与利用率	3	3	4	4	3	5	3
V	计算与传输重叠	3	3	4	4	3	5	3
W	多卡通信	3	3	4	4	3	4	3
调试								
E	错误码排查	3	4	3	4	2	4	3
X	内存越界	2	3	4	4	2	4	3
迁移								
Y	跨芯片迁移	3	3	4	4	3	5	3
Z	编程概念对照	3	3	4	4	4	5	3
迁移类比 (不计入 CANN/CUDA 汇总)								
G	迁移类比	3	3	4	4	3	4	3

M1–M10 为有序评分（1–5；高值更有利；M8 为逆向成本评分）。“—”表示官方正文详尽度未观测。

* M7 为模型自带知识估计。† M11 为由 M1–M8 汇总的综合置信度。G 的 CUDA 实为 ROCm/HIP，且不计入 CANN/CUDA 汇总。

Fig. 7. 完整指标矩阵 (/ 3)：按相同任务和工作流顺序呈现 M5–M8，即第三方来源数量、第三方来源可信度、模型自带知识估计和检索与获取成本。

Table 4. 表 四项指标的描述性统计。评分范围为–5分，SD为样本标准差。M3的CANN侧 $n = 24$ ，不含正文受阻的D；CUDA侧 $n = 25$ 。其余指标两侧均为 $n = 25$ 。M8分数越高，表示检索与获取成本越低。均值与标准差用于描述有序评分的分布。

指标	CANN $\bar{x}$ (SD)	CUDA $\bar{x}$ (SD)
M3 官方正文详尽度	4.21 (.83)	4.80 (.41)

指标	CANN $\bar{x}$ (SD)	CUDA $\bar{x}$ (SD)
M4 资料版本清晰度	2.88 (.01)	4.24 (.83)
M8 检索与获取成本	3.24 (.36)	3.88 (.51)
M10 回答步骤可操作性	4.00 (.76)	4.72 (.68)

6 发现：用跨生态对照定位知识条件

任务级比较揭示了三个相互关联的发现。首先，两侧生态在不同开发场景中的知识支撑呈现不同分布。其次，具体任务中的差异涉及官方资料的发现、交付、内容详尽程度及版本组织。最后，第三方资料与模型自带知识的支撑也因任务而异，需要结合官方材料考察。下面依次呈现场景对比、具体知识缺口及来源之间的互补情况。CANN/CUDA的差异用于暴露诊断线索：它们描述本任务集和访问配置下的条件，不构成生态整体能力排名，也不是本文核心构念的替代物。

6.0.1 从差异到诊断：根因分类与不确定性. 跨生态差异本身不是根因。只有把任务需求与搜索结果、返回形态、关键来源说法和最终答案连接起来，才能提出更具体的诊断；否则应把解释保留为待核验假设。

观测组合	可以支持的 诊断方向	不能直接推出的结论	下一步核验或改进
M1低，且保存的搜索列表没有相关官方命中	本次查询和预算下官方入口未被发现	文档不存在或生态缺少相关知识	重查查询词、官方索引和已知目标的直接获取
M1高但M2仅返回框架、摘要或不完整正文	入口可见，但正文交付或表示形态存在问题	正文一定缺失，或页面技术类型本身就是原因	检查返回表示、正文边界、重试和同文档的其他官方出口
M2可评但M3低	已取得的官方正文没有覆盖冻结任务中的步骤、参数或约束	整个生态都缺少该知识	将缺口映射到任务需求，并核查对应版本章节
M2/M3较高但M4低	操作内容存在，但必要的版本—组件适用关系缺失	相关版本必然不兼容	查找兼容表、适用条件和冲突，并检查答案是否臆造关系
M5高但M6低	来源数量不等于关键说法可核验	未核验说法一定为假	保留原子说法，补充独立核验和来源归属证据

观测组合	可以支持的 诊断方向	不能直接推出的结论	下一步核验或改进
M7低而外部资料与最终答案较强	本题无检索 先验不足， 外部获取部 分补偿	训练数据密度或生态文 档质量	保留冻结先验与外部证 据，做同模型重复或专 家核查
来源支撑充分而M9/M10低	知识可得， 但回答合成 中的定版或 操作组织失 败	缺陷一定来自文档	逐条回溯回答主张、版 本条件、代码和修正清 单

因此，“没有找到”“没有取得”“没有识别”和“文档没有提供”不是同义词。任务起点和范围、模型能力与检索策略、搜索提供方与预算、页面返回形态、来源归属、评分判断都会影响观察；本批每个模型每侧只有一次有效运行，不能把条件化差异写成生态因果结论。每项修复建议都应同时给出下一项验证，例如重查官方目标、补充独立来源核验、进行同模型重复，或在可行时执行操作。

6.1 不同生态在开发场景中的知识支撑分布存在差异

按 workflow 比较，CANN 的环境与安装任务综合置信度均值为 $\bar{x} = 0.799$ ，算子开发与调试分别为 $\bar{x} = 0.658$ 和 $\bar{x} = 0.646$ ；CUDA 对应均值分别为 $\bar{x} = 0.904$ 、 $\bar{x} = 0.891$ 和 $\bar{x} = 0.933$ （表 5）。调试组的两侧差值最大，为 .287。比较因此将关注点引向具体开发活动：在本次任务集中，CANN 对环境配置的知识支撑较充分，而算子开发和调试面对较弱的支撑条件。

Table 5. 表 5 对任务的各 workflow 的综合置信度均值。差值为 CUDA 减 CANN，使用未舍入的均值计算后再保留三位小数；不含 G。

workflow	任务数	CANN $\bar{x}$	CUDA $\bar{x}$	差值
环境与安装	3	0.799	0.904	0.106
算子开发	6	0.658	0.891	0.233
训练	4	0.791	0.945	0.154
推理与部署	4	0.722	0.920	0.198
性能与优化	4	0.752	0.957	0.205
调试	2	0.646	0.933	0.287
迁移	2	0.787	0.921	0.133

C. 回答检查与综合置信度

按 workflow 组织的指标评分。每项任务均列 CANN 与 CUDA；G 的第二列为 ROCm/HIP。

任务	M9 回答版本		M10 步骤可操作		M11 综合置信度†	
	CANN	CUDA	CANN	CUDA	CANN	CUDA
环境与安装						
I 版本配套	5	5	5	5	0.73	0.85
J 安装与环境	4	4	5	5	0.80	0.88
K 容器环境	4	4	5	5	0.86	0.98
算子开发						
C 算子库选型	4	5	4	5	0.75	1.00
D 自定义算子	2	5	3	5	0.41	0.88
L 算子精度	3	4	4	4	0.69	0.84
M 动态形状 / Tiling	4	4	4	5	0.63	0.94
N 算子融合	4	4	4	4	0.74	0.83
O 注册与框架集成	4	4	5	5	0.72	0.84
训练						
F 分布式训练	3	4	3	5	0.72	0.88
P 混合精度	4	5	4	5	0.83	0.98
Q 显存 / OOM	3	4	4	5	0.86	0.94
R 训练精度与收敛	3	4	4	4	0.75	0.98
推理与部署						
A 模型转换 / 导出	3	5	4	5	0.75	0.94
H 量化	4	4	5	5	0.69	0.88
S 推理部署	4	4	4	5	0.74	0.94
T 动态推理	4	5	4	5	0.72	0.92
性能与优化						
B 性能定位	3	4	3	5	0.70	0.93
U 访存与利用率	4	4	4	4	0.88	1.00
V 计算与传输重叠	4	5	4	5	0.72	0.98
W 多卡通信	4	5	4	5	0.70	0.92
调试						
E 错误码排查	3	4	3	5	0.55	0.99
X 内存越界	4	4	5	5	0.74	0.88
迁移						
Y 跨芯片迁移	4	5	4	5	0.67	0.92
Z 编程概念对照	2	2	2	2	0.90	0.92
迁移类比 (不计入 CANN/CUDA 汇总)						
G 迁移类比	4	4	4	5	0.84	0.88

M1–M10 为有序评分 (1–5；高值更有利；M8 为逆向成本评分)。“—”表示官方正文详尽度未观测。

* M7 为模型自带知识估计。† M11 为由 M1–M8 汇总的综合置信度。G 的 CUDA 实为 ROCm/HIP，且不计入 CANN/CUDA 汇总。

Fig. 8. 完整指标矩阵 (/3)：按相同任务和工作流顺序呈现 M9–M11，即回答版本明确性、回答步骤可操作性和由 M1–M8 汇总的综合置信度。

Table 6. 表 对任务按开发场景分组的综合置信度均值与样本标准差。上手与环境组为A、H、I、J、K、S、T、Y、Z；训练与日常使用组为E、F、P、Q、R；算子与深入自研组为B、C、D、L、M、N、O、U、V、W、X。两侧归组相同，不含G。

开发场景	任务对数	CANN $\bar{x}$ (SD)	CUDA $\bar{x}$ (SD)
上手与环境	9	0.762 (.078)	0.915 (.038)

开发场景	任务对数	CANN $\bar{x}$ (SD)	CUDA $\bar{x}$ (SD)
训练与日常使用	5	0.743 (.121)	0.953 (.045)
算子与深入自研	11	0.699 (.113)	0.914 (.062)

场景分组进一步显示，CANN的综合置信度均值从上手与环境组的 $\bar{x} = 0.762$ ，降至训练与日常使用组的 $\bar{x} = 0.743$ ，再降至算子与深入自研组的 $\bar{x} = 0.699$ ；CUDA三组均值分别为 $\bar{x} = 0.915$ 、 $\bar{x} = 0.953$ 和 $\bar{x} = 0.914$ ，未呈现同样的下降模式。这一对比识别了CANN在深入自研场景中的知识支撑短板，使评价能够形成关于生态如何支持不同开发活动的具体认识。

组内案例进一步说明这些场景所需的知识。CANN的Tiling任务M和自定义算子任务D分别得分.628和.410；深入自研组中的访存与利用率任务U则为.881，取得了官方最佳实践材料。训练与日常使用组中的错误码任务E仅为.554，其取得的官方解释缺少诊断细节。下面结合这些任务的资料获取与内容记录，分析场景差异所涉及的知识供给问题。

6.2 官方资料的发现、获取、内容与版本组织呈现不同缺口

官方资料的可发现性也存在差异。对任务中，CANN与CUDA的M1均值分别为 $\bar{x} = 3.56$ 和 $\bar{x} = 4.40$ ，使用两轮搜索的任务分别为项和项。在A任务中，CANN的首条官方结果位于第位，对应CUDA任务的官方结果位于第位。这些差异体现了官方资料在任务相关搜索中的呈现位置，以及检索过程所需轮次的不同。

官方资料被找到后，还需检查任务所需的正文是否实际取得。主比较中，CANN的项任务有项取得核心正文、项取得部分内容、项未取得核心正文；CUDA的项任务均取得核心正文。多数任务已获得官方内容，获取障碍集中于少数路径。对于这些路径，需要结合任务要求检查缺少的是哪些操作材料。

任务D要求创建Ascend C算子并将其与框架集成，需要实现代码、配置字段说明及编译部署步骤。在D任务中，Agent通过web_fetch读取官方Ascend C开发页面时，只取得了导航、简介和文档链接等页面框架信息，没有取得实现步骤、配置说明及编译部署正文。web_fetch明确报告“No code blocks, no implementation details, no step-by-step commands”（没有代码块、实现细节或分步命令）。对应的CUDA任务则取得了详尽官方正文，M3为。M1记录官方来源的发现条件，M2记录获取结果，D的CANN侧M3标记为“受阻”。这一差异提示，官方页面的内容交付方式需要支持Agent实际读取操作正文，使已提供的技术资料能够成为回答的依据。

正文取得之后，其对任务的支撑仍有差异。CANN的条可评价记录中，M3为分和分的各有条，另有条为至分；CUDA的条记录均为至分。在E的CANN任务中，Agent通过web_fetch从官方EZ9999错误说明页取得的内容中，原因栏标为“N/A”，处理建议包括“检查CANN安装包是否正确”和“检查环境变量是否正确”，但未提供具体的环境变量名称、检查命令或结果判断方式。Agent此次取得的内容仅包含通用检查建议，缺少针对该错误的诊断信息，难以据此确定下一步操作，其M3为。对应CUDA任务取得的检测工具指南包含具体命令、工具选项及报告格式，M3为。这里的差异涉及实际取得的材料能否提供诊断所需的步骤与解释。

同属调试场景的X则在两侧均取得详尽官方指导，M3均为。E与X的比较进一步将问题定位到具体任务内容：CANN的内存检测任务已有较充分的官方指导，而错误码诊断任务仍缺少选择下一步检查所需的信息。相应的改进对象是错误解释与诊断内容的完善。

任务A展示了可用主路径与不可得参考资料并存的情况。Agent通过web_fetch读取官方转换快速入门时，取得了ATC转换命令及参数解释，其中包括设置输入形状的--input_shape、指定目标处理器的--soc_version，以及通过npu-smi info查询处理器信息的指导。这些内容为转换命令及其参数填写提供了依据；更完整的ATC/AIPP参考页则仅返回导航和元数据。填写示例将快速入门关联到获得支持的转换需求，将缺失参考关联到尚未解决的高级设置。任务Y通过文档中心获取跨芯片迁移信息时，同样仅取得部分内容。这些案例说明，记录应同时保留

读取状态辅助剖面

该辅助图展示正文获取状态，补充图2–4中的 M2 官方正文可获得性评分。

任务	正文读取状态		官方正文详尽度		官方正文清晰度
	CANN	CUDA	CANN	CUDA	
环境与安装					
I 版本配套	C	C	4	5	4
J 安装与环境	C	C	5	5	2
K 容器环境	C	C	5	4	4
算子开发					
C 算子库选型	P	C	3	5	3
D 自定义算子	N	C	—	5	2
L 算子精度	C	C	5	5	2
M 动态形状 / Tiling	C	C	4	5	4
N 算子融合	C	C	5	4	2
O 注册与框架集成	C	C	5	5	3
训练					
F 分布式训练	C	C	4	5	2
P 混合精度	C	C	4	5	4
Q 显存 / OOM	C	C	5	5	4
R 训练精度与收敛	C	C	5	4	3
推理与部署					
A 模型转换 / 导出	C	C	4	5	2
H 量化	C	C	3	5	2
S 推理部署	C	C	4	5	3
T 动态推理	C	C	5	5	2
性能与优化					
B 性能定位	C	C	4	5	2
U 访存与利用率	C	C	4	5	5
V 计算与传输重叠	C	C	4	5	2
W 多卡通信	C	C	3	4	3
调试					
E 错误码排查	C	C	2	5	2
X 内存越界	C	C	5	5	2
迁移					
Y 跨芯片迁移	P	C	4	5	3
Z 编程概念对照	C	C	5	4	5
迁移类比（不计入 CANN/CUDA 汇总）					
G 迁移类比 [CANN / ROCm-HIP]	C	C	4	5	4

内容状态：C=核心正文已取得；P=部分内容已取得；N=未取得。访问路径在协议中单独记录。

G 的 CUDA 实为 ROCm/HIP；它是迁移类比，不计入 CANN/CUDA 汇总。

Fig. 9. 读取状态辅助剖面：将正文获取状态与官方正文详尽度、资料版本清晰度评分并置，沿用图–8的工作流分组和任务顺序，辅助解释获取结果。

取得了什么内容、它与哪项任务需求相关，并在有记录依据时单独说明访问路径。对于正在判断转换指导的开发者，这一区分标出了已有材料支持的主流程与仍需参考资料的高级设置，也限定了所取得来源能够为建议中的哪些操作提供依据。

补充材料中的“Selected web_fetch evidence / web_fetch证据摘录”列出A快速入门、D开发入口和E错误条目的读取URL、时间、请求及相关返回片段。这些摘录呈现了Agent在各次读取中实际收到的web_fetch内容，为上述案例分析提供具体依据。

即使取得了核心正文，获取成本仍可能不同。K的两侧均取得官方核心正文，但CANN记录了四次抓取和一次抓取失败，CUDA为两次抓取且无失败，M8分别为和。这些记录将资料入口及获取路径中的额外尝试，纳入了对知识供给条件的分析。

版本组织涉及更广的任务范围。CANN的M4均值为 $\bar{x} = 2.88$ ，CUDA为 $\bar{x} = 4.24$ ；CANN有项任务为分，分布于转换、安装、训练、算子开发、推理、优化与调试。A中相似快速入门材料分布于不同版本树，取得正文后仍需要确定应采用哪一版，其M4为；对应CUDA任务为。这类差异将关注点引向版本推荐、适用范围及配套依据的组织。

I则在两侧均取得了配套信息，M4均为，所形成指导的M9均为。CANN任务需要结合工具包、驱动、框架与集成包等来源，取得的配套矩阵为连接这些关系提供了依据。这说明版本评价既能够识别选择依据不足的情况，也能够记录已有配套说明如何支撑任务。对生态建设而言，需要明确的是各项资料之间可供查询和核验的关系。

检索委托给Agent后，还需判断资料声明的适用条件是否与开发者的设备和软件组合相符。来源适用范围与本地环境事实分别提供这一比较所需的信息：缺少配套关系时需要查证来源，未说明已安装版本时则需要检查本地环境。

6.3 不同来源对任务的补充作用与局限

在对任务中，CANN与CUDA每项任务检索到的第三方候选来源均值分别为 $\bar{x} = 3.16$ 和 $\bar{x} = 3.44$ 条；仅有至条候选来源的任务分别为项和项。第三方资料的补充条件需要结合具体任务分析。在X中，两侧均取得详尽官方指导，第三方候选来源分别为条和条；I则分别为条和条。数量上的差异并非在所有任务中都朝同一方向出现。

来源记录进一步显示，候选条目还受到平台分布和获取条件的影响。例如，K的CANN侧两条候选来源均来自CSDN，CUDA侧四条来源分布于不同平台；N、O、Q的CANN记录包含知乎正文获取受阻的情况。平台分布有助于确定需要检查哪些来源关系，实际返回内容则决定这些条目能否提供可用说明。同一平台上的不同文章是否独立，仍需结合其内容判断。

来源组合使官方与社区知识的互补情况更加具体。X的CANN任务虽然第三方候选来源有限，但已有详尽官方指导；D则缺少核心官方正文，第三方可信度评分也较低。E记录了在官方错误说明不足后继续寻找案例的过程。这些情况分别指向官方指导之外的补充经验、核心指南与替代支撑的共同不足，以及通过其他来源补充诊断信息的需求。生态知识建设因而需要关注各类任务中，哪些知识由官方资料提供，哪些仍依赖可发现、可取得的实践案例与独立解释。

模型自带知识的估计提供另一项支撑信息。主比较中，CANN与CUDA的M7均值分别为 $\bar{x} = 2.84$ 和 $\bar{x} = 4.56$ ；CANN的D、E、I、M、N、S、X为分，CUDA各项均为或分。将这一估计与来源记录并置，可以定位需要重点检查外部知识支撑的任务。本研究未单独测试无检索条件下的表现，因此，该估计用于描述知识支撑组合，不作为模型能够独立完成任务的验证。

M9与M10进一步记录任务中形成的指导。例如，X两侧的M10均为，E则分别为和。这些回答属性与实际取得的材料共同提供检查依据，使分析能够从知识来源延伸到所形成指导的版本说明与操作条件。

7 讨论：知识供给与 AGENT 交互的设计启示

上述比较从开发场景、具体知识缺口及来源互补三个层面，形成了对生态知识支撑的认识。下面据此讨论建设重点、官方资料与Agent反馈的改进，以及社区知识与来源核验。设计示意用于具体说明其中的潜在应用，尚未评价其实施效果。

7.1 根据开发场景与知识需求确定建设重点

本次场景对比将CANN深入自研任务中的知识支撑不足识别为一个需要关注的建设方向。进一步检查D、M等任务，可以将这一方向落实到实现指南、操作细节及补充案例等具体需求；E则说明诊断知识也需要作为独立任务内容建设。这体现了任务级研究的诊断价值：通过比较开发活动及其所需知识，形成关于生态建设重点的具体认识。

这些认识对应不同范围的改进。多个工作流中反复出现的版本问题，适合通过共享的推荐入口与配套关系组织加以处理；D中未取得的核心正文、E中不足的诊断细节，则分别需要改善内容交付和完善任务指导。第三方补充有限的任务还需要检查实践案例和独立解释的覆盖情况。改进成效可以回到原任务中，检查相关知识需求是否得到更充分的支持。

7.2 改善官方知识供给与Agent反馈

反复出现的版本问题提示，应将适用性作为文档系统的一项属性。稳定的推荐版本入口可以与索引清晰的旧版本文档并存，每页同时声明适用版本和验证日期。驱动、工具包、框架及集成包之间的兼容关系可以通过可查询形式发布。对于I这类任务，开发者或Agent就能够针对已说明的环境请求受支持组合，减少从多个分散记录中拼接配套关系的需要。

图将这一建议具体化为配套查询界面。设备与框架条件限定查询范围，返回的组合同时保留来源、适用范围和验证日期；未找到有依据的匹配时，界面呈现尚未确定的条件。这使M4所指向的版本问题成为可检查的信息关系，也为后续确认回答的适用性提供依据。

依赖版本的指导应将来源的适用范围与已有任务环境关联起来。在提出工具包/框架组合前，Agent可以利用任务中已提供的设备与软件事实，仅请求建立匹配所必需的其余信息。回答层指标随后检查建议是否说明适用版本条件，以及操作步骤是否明确了所需的替换或修改。

图以独立的环境记录说明版本核对：先区分开发者提供的本机事实与来源声明的适用范围，再将比较结果记录为匹配、不匹配或信息不足。环境记录在任务内复用，并随用户补充而更新，以支持具体命令建议前的适用性判断。

局部获取与内容缺口则需要针对任务修复。对于D所呈现的正文获取缺口，文档系统应确保Agent读取官方开发页面时能够取得完整操作正文。具体方式可以是确保HTML页面直接返回完整内容，也可以提供与网页同步更新的Markdown或纯文本版本。无论采用何种方式，都应保留任务所需的操作步骤、参数解释和适用版本，并通过相应任务检查Agent能否实际取得这些内容。E已经有独立错误页面，其改进需要充实该入口内的内容：错误语义、可能触发条件、诊断步骤、相关issue案例链接及适用版本。对于通用兜底错误，指南可以说明仅凭该错误码尚不能确定什么，并明确下一步需要哪些日志或本地观测。这样的页面围绕用户问题组织已有证据，以支持诊断。

图以E的错误页说明任务导向内容的组织方式：从错误码能够支持的判断出发，明确需要收集的现场信息，将可观察现象关联到下一步检查和有来源的案例，并为未解决情境保留支持入口。实际填入的原因、检查步骤与案例需要由领域维护者确认；该结构帮助定位哪些知识需要补充。

同样的任务导向结构可以用于其他文档单元。预期结果、前置条件、最小命令或代码、参数说明、预期输出和恢复信息，能够为人类阅读及Agent检索提供具体材料。机器可读导出与索引应保留这些关系及其版本范围。其效果可以依据最初促成改动的任务，检查相关内容获取和详尽度字段。

图展示如何让同一任务资料通过文档正文和机器可读出口提供。以任务A中取得的转换快速入门所提供的命令与参数关系为例，导出同时保留任务、适用版本、来源、参数解释和关联步骤。对于D这类缺失核心正文的



Fig. 10. 版本与配套查询的设计示意，依据 A、I 中分散的版本资料提出。选择条件、结果字段与来源检查入口展示拟议的信息组织方式；配套值为占位字段。

路径，目标是让相关指导实际送达；对于已有可读正文的路径，目标是避免导出时丢失上下文。验收应比较取得内容与任务要求的对应关系，而不只检查是否存在某种文件格式。

Agent 界面可以保留找到相关来源、取得正文和支持某个操作步骤之间的区别。对于 D，有用的状态信息应指出缺失的实现指南；对于 E，应说明官方解释较为笼统，并请求进一步诊断所需的日志或触发情境；对于 A，可以保留可用转换路径，同时标明尚未解决的高级参考设置。这些状态将一次检索事件连接到它对开发者当前任务的影响。

1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456

先明确环境，再核对建议

当前任务的环境记录

设备型号	工具包版本	框架与集成包
<device>	<local version>	<local version>

来源材料的适用范围

版本 / 组件 / 兼容条件：由引用材料提供

当前状态：尚未完成环境匹配

请补充缺失版本；已知不匹配的材料不直接用于当前命令建议。

核对结果可分别记录：匹配 / 不匹配 / 信息不足。

更新任务环境

依据 A / I 的版本条件；字段为示意。来源范围与本机事实分别记录，关联 M4、M9。

Fig. 11. 环境声明与版本核对的设计示意，依据 A、I 的版本条件提出。设备、工具包及框架字段代表用户提供的本机事实，来源范围独立记录；图中展示信息尚不足以建立匹配的状态。

图对照D与E的拟议响应。对于D，Agent先寻找正文链接或其他可读来源，并保留原始入口供开发者按需查看；对于E，Agent说明触发情境与相关日志片段如何用于查找诊断案例。开发者补入内容作为获取失败后的备选途径，核心指南的可获取性仍应由知识提供方维护。M1–M3定位缺口，M8记录后续尝试的成本。

知识供给的改进还涉及入口与维护。稳定且可检索的官方入口影响候选资料能否被发现；明确来源归属、修订日期及兼容关系则影响后续判断。团队可以将代表性任务作为持续检查单元：跟踪查询能否命中目标资料、

EZ9999：诊断资料的组织方式

任务 E 中观察到：已有专属错误页，原因与处理建议泛化。

诊断起点：错误码本身不足以确定具体原因。
适用范围：<版本 / 组件> 更新依据：<来源 / 日期>

1 明确需要收集的现场信息

报错前后的日志、触发操作、相关组件与实际版本。
标明每项信息用于区分哪类情境。

2 按可观察现象组织排查路径

现象 / 条件	下一步检查	案例与适用依据
<日志特征 / 触发情境>	<检查步骤与预期观测>	<案例链接 / 版本>

3 保留未解决分支与后续支持入口

未能定位时，说明还缺哪些证据，并带上已有记录继续求助。

示例依据：E；对应 M3、M4。排查结构为建议，案例字段为占位。

Fig. 12. 面向任务的诊断内容示意，依据 E 中“已有错误页但指导泛化”的发现提出。图中组织诊断起点、现场信息、检查路径、案例依据与未决分支；具体案例和版本范围为待填字段。

正文是否完整送达、版本关系是否变化，以及社区案例是否仍适用。这样的维护机制也适用于没有专门界面示例的来源治理与第三方知识供给。

针对搜索可见性的差异，文档团队可检查官方页面的搜索引擎收录情况，以及标题、摘要与开发任务用语的匹配程度，并以任务查询下的官方结果位置和发现所需轮次检验SEO改进。

同一任务资料，保留关系的两种出口



结构示意；命令与参数依据 A。导出应保留正文、版本及引用关系，对应 M2-M4。

Fig. 13. 正文与机器可读出口的结构示意。命令、参数及设备查询关系来自任务A中取得的转换快速入门；结构化文本是拟议导出形式。两种出口保留同一任务的版本、来源与步骤关系。

7.3 扩充社区知识并支持来源核验

社区知识建设可以围绕任务中尚未充分获得支持的情境展开，例如具体故障案例、不同配置下的操作经验及替代实现方式。这一方向与利用Stack Overflow补充API文档的研究相呼应 [34]；本文进一步关注这些补充材料能否被Agent发现和取得，以及它们能否支撑具体任务需求。对于候选来源较少的任务，应检查缺少哪些补充说明；对于来源集中或正文无法取得的任务，应检查独立实践依据与内容交付条件。建设目标是使任务所需的经验能够被发现、取得并明确归属，而不仅是增加内容条目。

两类知识缺口，两种后续行动

(a) 正文未取得 · 基于任务 D

- 1 找到相关官方指南
- 2 返回了导航与元数据
- 3 核心操作指南正文未取得

缺口：实现步骤缺少官方正文支撑

我已取得相关资料，但尚不能据此核对这份指南中的完整实现过程。

我会先查找正文链接或可读来源，你也可以按需查看原始指南。

[按需查看原始指南](#)

(b) 诊断内容不足 · 基于任务 E

- 1 找到官方 EZ9999 页面
- 2 页面正文已取得
- 3 仅有通用诊断建议

缺口：无法据此缩小报错原因

这页未给出具体原因，请补充触发情境与相关日志片段。

我会结合这些信息查找相关案例并建议下一步诊断检查。

[补充情境与日志](#)

基于任务 D / E 的观察。界面消息为设计示意；对应 M1-M3，重试过程关联 M8。

Fig. 14. 两类知识缺口的 Agent 界面设计示意：(a) 基于 D 的核心正文未取得；(b) 基于 E 的正文可读但诊断内容不足。问题条件来自任务观察，界面消息与后续动作是拟议设计。

呼应人-AI交互指南中解释系统行为与支持纠错的建议[3]，Agent可以通过保留来源段落、适用条件及其与建议的关系，帮助开发者检查已有依据。对于仍然缺少的内容，应说明尚未获得支持的任务部分，并据此继续查找或请求必要的现场信息。来源核验与纠错反馈示意具体展示了这种信息关联方式。

由于解释可能提高建议采纳率，却未改善人机协作表现[4]，来源展示的效果应通过开发者能否识别缺少依据或不适用的建议来评价。

1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664

核验转换指导

当前任务：模型转换

回答、来源与本次反馈保持关联

回答中的转换命令（节选）

atc --model=resnet50.onnx --framework=5 ...

官方快速入门 · CANN 8.0.RC2

展开来源段落

atc --model=resnet50.onnx --framework=5 --output=resnet50
--input_shape="actual_input_1:1,3,224,224" --soc_version=<soc_version>

来源入口：官方模型转换快速入门

前往原始页面

这条指导与你观察到的情况不一致？

关联本条命令提交报错或修正，保留原来源与任务环境。

下一轮检查将同时使用原建议与新增反馈。

添加报错或修正

命令节选来自任务 A 中取得的快速入门；界面交互为示意。对应 M4、M9、M10。

Fig. 15. 来源核验与纠错反馈的设计示意。转换命令节选来自任务A中取得的快速入门；来源展开、原文入口及关联原建议的报错反馈是拟议交互。

图 15 进一步示例来源核验与纠错反馈。开发者从命令展开引用段落，检查适用版本，并将实际报错或修正关联到原建议。原来源、任务环境与新增反馈共同保留，使下一轮检查能够聚焦具体分歧。这为 M9、M10 所描述的回答属性提供了可检查的依据，并支持后续修订。

这些建议将核验建议视为编程工作的一部分，与前文讨论的信息寻找及AI辅助研究相衔接 [5, 7, 36]。展示每次检索事件和全部来源段落，可能增加阅读负担，却未必使下一步行动更清楚。界面可以先呈现获得支持的步

骤、相关版本范围与未决条件，再按需展开来源段落。这样既优先呈现当前判断所需的信息，也保留深入检查的入口。

请求环境事实也可能打断任务，尤其是在重复询问已经提供的信息时。每次请求应说明哪项建议依赖这一事实，并复用已有任务上下文；信息可能过时再请开发者确认。对于诊断日志，应请求相关片段，并允许移除敏感内容。缺失的配套关系和笼统的错误解释需要文档与系统团队完善，收集更多本地事实本身不能补足这些解释。如何平衡核验、打断与有效指导，仍需在这些建议的后续评价中考察。

7.4 诊断方法的复用

该方法将任务需求、来源或获取事件、指标编码，与判断技术建议所需信息的诊断连接起来。参与开发者平台、技术知识资源或AI Agent建设与维护的产品经理、设计师、技术写作者和开发工程师等，可据此定位缺失的指导或适用关系，并区分需要继续查找来源的情形与需要补充本地事实的情形。分析者可以沿用记录结构，并替换领域特定需求，例如数据库SDK中的客户端与服务端兼容性，或Web框架的部署前置条件。这些构成方法的预期应用方向。

后续评估可以考察独立分析者是否形成相近剖面、诊断是否与专家对缺失信息的判断一致，以及文档团队是否认为它们有助于规划修复。这些问题直接来自方法的预期用途。本次应用提供任务级区别、填写示例与跨任务综合分析，为开展此类评估奠定基础。

8 局限与研究透明度

本次三模型试跑检查了三个任务上的规则执行与证据链；每格仅一次，任务不是总体随机样本，也不是新留出任务，两生态题目不保证等难。历史审计、系数拟合批次和本次试跑使用不同目的与条件，不能合并为全任务多模型验证。模型别名不保证底层快照固定，搜索提供方未独立锁定，检索工具的差异与模型表现交织。

所有运行采用统一后评流程。原始评分、复核评分及未解决的引文和适用性问题分别保留；逐字引文检查与文件/预算检查不证明技术正确性，更不构成独立专家验证。M2的部分完整性依据为事后参照；E的M4范围遗漏使对应比较暂缓。多数记录无法计算M11点值，不能删掉缺失项或仅根据可计算记录推断生态或模型优势。量规的一致性、用途效度和系数的适用性仍需另外评价。

未进行硬件执行，不能报告代码或命令实际复现成功。开发者如何使用这些证据判断本地适用性，以及拟议反馈对阅读负担、任务打断和实际完成情况的影响，仍需用户研究。文献提供概念与方法依据，不验证本文具体五档阈值和系数。

9 结论

本文的核心贡献是把面向AI的知识可得性定义为可检查的任务关系，并用十一项指标把知识来源、获取过程、证据支撑和最终回答连接起来。它服务于文档团队和Agent建设者：帮助他们定位资料未被发现、正文未被取得、关键任务内容缺失、版本关系不清或回答仍需补改的具体环节。CANN/CUDA案例提供了一个压力测试场景：任务级比较识别了不同开发场景中的知识支撑差异，并进一步揭示了正文交付、诊断内容、版本组织及补充来源方面的具体条件；这些差异是本任务集和访问配置下的诊断输出，不能脱离任务范围解释为生态整体排名。将评分与任务需求、实际取得的材料联系起来，使研究能够形成关于生态如何支持开发活动，以及哪些知识需要完善的具体认识。

该方法使经由AI形成的技术指导的依据成为可检查的对象：哪项来源支撑拟议步骤，哪项适用关系尚未明确，以及还需要什么信息。框架、协议与案例分析提供了定位这些条件，并将其关联到知识供给和Agent反馈的方法。文档修复、继续检索和请求本地事实分别对应不同的诊断，为后续评价提供具体方向。

附录A：评分计算细则

以下M1–M10分档与判定边界对应本次试跑的冻结规则；每项五档为研究约定，文献不直接验证具体档位。分项原始记录保留未评价、缺测、不适用、受阻和证据不足等状态。

M1 官方可发现性

1. 在预先规定的统一搜索预算内，未发现相关官方来源
2. 第三次或之后的查询才首次发现相关官方来源
3. 第二次查询才首次发现相关官方来源
4. 首次查询第—5条出现相关官方来源
5. 首次查询第条即为相关官方来源

试用协议每次观察前条，最多次查询；预算为实验条件，不是通用评分定义。查询而非调用计数，批量与并行须统一协议。首次命中后继续检索不降分，改写本身不另扣分。前序超时、列表缺失或未完成预算且未命中记待定。保存每轮全部网址与精确排名。此五档是研究约定，不是已验证量表。

M2 官方可抓取性

1. 未取得可识别的目标页面内容：请求失败、拒绝访问、错误页或无返回
2. 取得目标页面标题、导航或元数据，但没有可识别正文
3. 只取得明确标注为摘要或摘录的正文表示，未取得直接正文返回
4. 取得直接正文，但工具明确标注截断、缺页，或可核对的正文边界存在缺失
5. 取得直接正文，且未观察到上述截断、缺页或目标边界缺失

按每个独立官方文档的最终获取状态判定。先区分无内容、框架、摘要/摘录、直接正文，再核对是否观察到截断、缺页或目标边界缺失。摘要中含代码不自动升级；工具自称完整提取不能独自证明全文完整。分只表示在允许的记录和核验材料中未观察到这些缺损，完整性另以独立核验状态记录，未知本身不把分数变成—5。获取目标须在读取前明确，不能事后缩小成恰好返回的段落。复合返回如完整正文加摘要按正文证据判定；整份返回为摘要且含原文片段仍属分。M3评内容是否详尽，M4评版本；不借答案质量判断获取状态。不同独立文档等权平均，保留原始均值；重试不重复计入，未知项不能删除后平均。

M3 官方正文详尽度

1. 取得的正文只有概述，没有相关解释或操作细节
2. 有相关细节，但只有零散参数、片段或说明，不能组成完整解释或操作段落
3. 有完整解释或操作段落，但完成原题所需的关键步骤、参数或约束仍缺失
4. 原题所需主流程及其参数、前提已说明，但原题明确涉及的分支、约束或参考细节未说明完整
5. 原题要求的解释、步骤、参数和约束均说明完整，无需另查才能补齐这些内容

先从原题列出需要说明的内容，禁止添加题外的穷尽要求；每项缺口附正文位置。按最高满足档判定。取不到正文记受阻，不给分；仅导航或元数据归M2。版本选择归M4。文档内容错误另记，不能只因长而判详尽。

M4 版本清晰度

1. 无版本信息
2. 有版本，但无选择依据，或官方说明冲突且无法消解
3. 已明确部分适用关系，但仍缺少完成版本选择所需的对应关系
4. 需自行合并官方约束，才能确定适用版本
5. 官方已明确适用关系，直接查表或套用规则即可确定

评价对象是已取得的官方资料；无正文可评记blocked，任务无需版本选择记not_applicable。只核对原题需要的版本条件，不增加题外要求。可选版本是一个或一个范围均可，不按页面数量或版本数量扣分。合并约束必须有官方依据，不得补入未说明的假设。记录版本条件、缺失关系与冲突的原文位置。检索成本归M8，最终答案定版归M9。无法区分档位时保留区间及原因，不强行猜分。

M5 二手丰富度

1. 相关独立第三方来源条
2. 相关独立第三方来源—2条
3. 相关独立第三方来源—4条
4. 相关独立第三方来源条
5. 相关独立第三方来源至少条

恢复原数量阈值，统一搜索预算内对相关来源去重计数，正文取得不是计数前提。官方仓库、官方论坛按已冻结的渠道分类排除；同一内容的转载或镜像只计一次。相关性无法确定则计数区间，区间跨档则待定。另报首次发现位置和查询数，避免声称数量单独涵盖全部可发现度。

M6 二手可信度 / 一致性

1. 存在与可靠核对依据直接矛盾的关键说法，尚未解决
2. 未发现已确证的关键矛盾，但关键说法没有可追溯的核对依据
3. 有可追溯依据支持部分关键说法，其他关键说法尚未核验
4. 全部关键说法均有可追溯依据支持，未见未解决矛盾，但缺少独立来源的交叉印证
5. 全部关键说法均有可追溯依据支持，并获独立来源交叉印证，未见未解决矛盾

关键说法限于会改变操作、解释或版本判断的可核验陈述，逐条记录支持/反驳/未知。权威性依据作者身份、第一手材料或可复核实验证据，不按平台名直接给分。转载不构成独立互证；旧文只有在内容已失效时才作为反证。没有第三方材料记N/A。此五档是证据可信程度量规，尚未校准。

M7 模型自带知识

1. 检索前回答没有可核验的正确任务知识，或关键主张被核对证据否定
2. 检索前仅能给出正确概念，无法给出具体方法或解释
3. 检索前能给出正确方法或解释片段，但关键细节需要外部资料补充
4. 检索前能够给出正确的完整流程或解释，原题涉及的具体参数、约束仍需补充
5. 检索前回答已准确覆盖原题所需的流程、解释、参数与约束

执行同题无工具回答并冻结，再独立核对其中知识正确性和完整性。回答前不能接触本轮检索内容；自评信心仅存记录，不用于计分。没有检索前回答的旧运行记缺测。该测量只能反映无检索知识表现，不能推断训练数据覆盖密度；未知正确性则待定。

M8 检索成本

1. $C \geq 9$
2. $C = 7-8$
3. $C = 5-6$
4. $C = 3-4$
5. $C = 1-2$

保留此前 $C=S+F$ 及五档，失败请求只按一次实际调用计数，不另罚。程序记录并强制相同预算；保存耗时及可得token。 $C=0$ 或异常中断不自动给高分。调用数不是完整计算成本。

M9 版本可锁定性

1. 最终答案未提供原题需要的版本信息
2. 提到版本，但没有给出有依据的适用版本或范围，或关键定版与证据矛盾
3. 部分所需组件有明确版本依据，其余仍未确定
4. 所有所需组件均给出有依据的适用范围，但至少一项尚未定到具体版本
5. 所有所需组件均给出有依据的具体版本，配套关系无未解决冲突

评价对象仅为最终答案。先固定原题实际需要定版的组件；不强制无关opset。符合证据的范围不等于答案错误，但锁定粒度低于具体版本。无版本需要记N/A；不能为提高分数随意指定版本。与M4区别：本项只看最终答案，官方资料清楚不代表答案已定版。

M10 操作可执行性

1. 没有可执行方案，只有目标描述或示意
2. 仅有操作骨架，关键操作无法根据答案确定
3. 有具体操作方案，但需修正错误或补充缺失内容才能执行
4. 操作指导完整，仅需替换明确标识且有说明的用户环境参数
5. 操作指导及适用前提明确，无需补写或修正内容即可直接执行

按最终答案中命令、代码、参数与前置条件判断需要补充或修改的程度。分仅替换路径、设备号等环境值，不能包括修正内容错误。分须指出具体缺口，分须说明哪项关键操作无法确定。每档附答案原文引用；实际运行通过、失败、仅内容核查或未验证另行记录，不把是否运行作为/5分界。仅内容核查不能宣称实际复现成功；证据不足记待定，纯概念任务记N/A。

M11 综合置信度

采用正文中的连续百分制公式： $x_i = M_i/5$ ， $OFF = x_1x_2x_3$ ， $SEC = x_5x_6$ ， $OWN = x_7$ ； $M11 = 100[1 - (1 - OFF)(1 - SEC)(1 - OWN)](0.7 + 0.3x_4)(0.9 + 0.1x_8)$ 。M9/M10独立报告。保留原始精度、展示一位小数，不转五档。当前计算器要求M1–M8全部为可用点值，未知、不适用或受阻不补零；未完成评价则M11待定。.30/0.10为工作基线，不宣称最优或概率校准。

REFERENCES

- [1] Emad Aghajani, Csaba Nagy, Olga Lucero Vega-Marquez, Mario Linares-Vasquez, Laura Moreno, Gabriele Bavota, and Michele Lanza. 2019. Software Documentation Issues Unveiled. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 1199–1210. <https://doi.org/10.1109/icse.2019.00122>
- [2] American Educational Research Association, American Psychological Association, and National Council on Measurement in Education. 2014. *Standards for Educational and Psychological Testing*. American Educational Research Association. https://www.testingstandards.net/uploads/7/6/6/4/76643089/standards_2014edition.pdf
- [3] Saleema Amershi, Dan Weld, Mihaela Vorvoreanu, Adam Fourney, Besmira Nushi, Penny Collisson, Jina Suh, Shamsi Iqbal, Paul N. Bennett, Kori Inkpen, Jaime Teevan, Ruth Kikin-Gil, and Eric Horvitz. 2019. Guidelines for Human-AI Interaction. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, 1–13. <https://doi.org/10.1145/3290605.3300233>
- [4] Gagan Bansal, Tongshuang Wu, Joyce Zhou, Raymond Fok, Besmira Nushi, Ece Kamar, Marco Tulio Ribeiro, and Daniel Weld. 2021. Does the Whole Exceed its Parts? The Effect of AI Explanations on Complementary Team Performance. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, 1–16. <https://doi.org/10.1145/3411764.3445717>
- [5] Shraddha Barke, Michael B. James, and Nadia Polikarpova. 2023. Grounded Copilot: How Programmers Interact with Code-Generating Models. *Proceedings of the ACM on Programming Languages* 7, OOPSLA1 (2023), 85–111. <https://doi.org/10.1145/3586030>
- [6] Godfred O. Boateng, Torsten B. Neilands, Edward A. Frongillo, Hugo R. Melgar-Quinonez, and Sera L. Young. 2018. Best Practices for Developing and Validating Scales for Health, Social, and Behavioral Research: A Primer. *Frontiers in Public Health* 6 (2018), 149. <https://doi.org/10.3389/fpubh.2018.00149>

- [7] Joel Brandt, Philip J. Guo, Joel Lewenstein, Mira Dontcheva, and Scott R. Klemmer. 2009. Two Studies of Opportunistic Programming: Interleaving Web Foraging, Learning, and Writing Code. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, 1589–1598. <https://doi.org/10.1145/1518701.1518944>
- [8] Shahul Es, Jithin James, Luis Espinosa Anke, and Steven Schockaert. 2024. RAGAs: Automated Evaluation of Retrieval Augmented Generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*. Association for Computational Linguistics, 150–158. <https://doi.org/10.18653/v1/2024.eacl-demo.16>
- [9] Claudia Flores-Saviaga, Benjamin V. Hanrahan, Kashif Imteyaz, Steven Clarke, and Saiph Savage. 2025. The Impact of Generative AI Coding Assistants on Developers Who Are Visually Impaired. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, 1–17. <https://doi.org/10.1145/3706598.3714008>
- [10] Tianyu Gao, Howard Yen, Jiatong Yu, and Danqi Chen. 2023. Enabling Large Language Models to Generate Text with Citations. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 6465–6488. <https://doi.org/10.18653/v1/2023.emnlp-main.398>
- [11] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. 2009. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (2 ed.). Springer. <https://hastie.su.domains/ElemStatLearn/>
- [12] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *The Twelfth International Conference on Learning Representations*. 54107–54157. https://proceeding.sicr.cc/paper_files/paper/2024/hash/edac78c3e300629acf6cbe9ca88fb84-Abstract-Conference.html
- [13] Saurav Kadavath et al. 2022. Language Models (Mostly) Know What They Know. arXiv:2207.05221 <https://arxiv.org/abs/2207.05221>
- [14] Diane Kelly. 2009. Methods for Evaluating Interactive Information Retrieval Systems with Users. *Foundations and Trends in Information Retrieval* 3, 1-2 (2009), 1–224. <https://doi.org/10.1561/15000000012>
- [15] Amy J. Ko, Robert DeLine, and Gina Venolia. 2007. Information Needs in Collocated Software Development Teams. In *29th International Conference on Software Engineering (ICSE’07)*. IEEE, 344–353. <https://doi.org/10.1109/ICSE.2007.45>
- [16] Christian Kohlschütter, Peter Fankhauser, and Wolfgang Nejdl. 2010. Boilerplate detection using shallow text features. In *Proceedings of the third ACM international conference on Web search and data mining*. ACM, 441–450. <https://doi.org/10.1145/1718487.1718542>
- [17] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, Vol. 33. 9459–9474. <https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>
- [18] Jenny T. Liang, Chenyang Yang, and Brad A. Myers. 2024. A Large-Scale Survey on the Usability of AI Programming Assistants: Successes and Challenges. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. Association for Computing Machinery, 1–13. <https://doi.org/10.1145/3597503.3608128>
- [19] Tianyang Liu, Canwen Xu, and Julian McAuley. 2024. RepoBench: Benchmarking Repository-Level Code Auto-Completion Systems. In *The Twelfth International Conference on Learning Representations*. <https://arxiv.org/abs/2306.03091>
- [20] Chang Ma, Junlei Zhang, Zhihao Zhu, Cheng Yang, Yujiu Yang, Yaohui Jin, Zhenzhong Lan, Lingpeng Kong, and Junxian He. 2024. AgentBoard: An Analytical Evaluation Board of Multi-turn LLM Agents. <https://doi.org/10.48550/arXiv.2401.13178> arXiv:2401.13178
- [21] Walid Maalej and Martin P. Robillard. 2013. Patterns of Knowledge in API Reference Documentation. *IEEE Transactions on Software Engineering* 39, 9 (2013), 1264–1282. <https://doi.org/10.1109/tse.2013.12>
- [22] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. 2008. *Introduction to Information Retrieval*. Cambridge University Press. <https://doi.org/10.1017/cbo9780511809071>
- [23] Sewon Min, Kalpesh Krishna, Xixi Lyu, Mike Lewis, Wen-tau Yih, Pang Koh, Mohit Iyyer, Luke Zettlemoyer, and Hannaneh Hajishirzi. 2023. FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 12076–12100. <https://doi.org/10.18653/v1/2023.emnlp-main.741>
- [24] Hussein Mozannar, Gagan Bansal, Adam Fourney, and Eric Horvitz. 2024. Reading Between the Lines: Modeling User Behavior and Costs in AI-Assisted Programming. In *Proceedings of the CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, 1–16. <https://doi.org/10.1145/3613904.3641936>
- [25] OECD, European Union, and Joint Research Centre - European Commission. 2008. *Handbook on Constructing Composite Indicators: Methodology and User Guide*. OECD. <https://doi.org/10.1787/9789264043466-en>
- [26] Neil Perry, Megha Srivastava, Deepak Kumar, and Dan Boneh. 2023. Do Users Write More Insecure Code with AI Assistants?. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. Association for Computing Machinery, 2785–2799. <https://doi.org/10.1145/3576915.3623157>
- [27] Peter Pirolli and Stuart Card. 1999. Information Foraging. *Psychological Review* 106, 4 (1999), 643–675. <https://doi.org/10.1037/0033-295X.106.4.643>
- [28] Y. Malini Reddy and Heidi Andrade. 2010. A Review of Rubric Use in Higher Education. *Assessment & Evaluation in Higher Education* 35, 4 (2010), 435–448. <https://doi.org/10.1080/02602930902862859>
- [29] Romain Robbes, Théo Matricon, Thomas Degueule, Andre Hora, and Stefano Zacchiroli. 2026. Agentic Much? Adoption of Coding Agents on GitHub. *ACM Transactions on Software Engineering and Methodology* (2026). <https://doi.org/10.1145/3822180> Advance online publication.

- [30] Martin P. Robillard. 2009. What Makes APIs Hard to Learn? Answers from Developers. *IEEE Software* 26, 6 (2009), 27–34. <https://doi.org/10.1109/MS.2009.193>
- [31] Steven I. Ross, Fernando Martinez, Stephanie Houde, Michael Muller, and Justin D. Weisz. 2023. The Programmer’s Assistant: Conversational Interaction with a Large Language Model for Software Development. In *Proceedings of the 28th International Conference on Intelligent User Interfaces*. Association for Computing Machinery, 491–514. <https://doi.org/10.1145/3581641.3584037>
- [32] Jon Saad-Falcon, Omar Khattab, Christopher Potts, and Matei Zaharia. 2024. ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. Association for Computational Linguistics, 338–354. <https://doi.org/10.18653/v1/2024.naacl-long.20>
- [33] J. Sillito, G. C. Murphy, and K. De Volder. 2008. Asking and Answering Questions during a Programming Change Task. *IEEE Transactions on Software Engineering* 34, 4 (2008), 434–451. <https://doi.org/10.1109/TSE.2008.26>
- [34] Christoph Treude and Martin P. Robillard. 2016. Augmenting API documentation with insights from stack overflow. In *Proceedings of the 38th International Conference on Software Engineering*. Association for Computing Machinery, 392–403. <https://doi.org/10.1145/2884781.2884800>
- [35] Gias Uddin and Martin P. Robillard. 2015. How API Documentation Fails. *IEEE Software* 32, 4 (2015), 68–75. <https://doi.org/10.1109/ms.2014.80>
- [36] Priyan Vaithilingam, Tianyi Zhang, and Elena L. Glassman. 2022. Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models. In *CHI Conference on Human Factors in Computing Systems Extended Abstracts*. Association for Computing Machinery, 1–7. <https://doi.org/10.1145/3491101.3519665>
- [37] Richard Y. Wang and Diane M. Strong. 1996. Beyond Accuracy: What Data Quality Means to Data Consumers. *Journal of Management Information Systems* 12, 4 (1996), 5–33. <https://doi.org/10.1080/07421222.1996.11518099>
- [38] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems*, Vol. 37. 50528–50652. <https://doi.org/10.52202/079017-1601>
- [39] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *The Eleventh International Conference on Learning Representations*. <https://arxiv.org/abs/2210.03629v3>
- [40] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. 2024. WebArena: A Realistic Web Environment for Building Autonomous Agents. In *The Twelfth International Conference on Learning Representations*. 15585–15606. https://proceedings.iclr.cc/paper_files/paper/2024/hash/4410c0711e9154a7a2d26f9b3816d1ef-Abstract-Conference.html